**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG**
**KHOA CÔNG NGHỆ THÔNG TIN 1**

# TIỂU LUẬN MÔN HỌC
# PHÁT TRIỂN HỆ THỐNG THÔNG MINH

| | |
|---|---|
| **Giảng viên hướng dẫn:** | PGS. TS. Trần Đình Quế |
| **Học phần:** | Phát triển Hệ thống Thông minh |
| **Sinh viên thực hiện:** | Vũ Thành Công |
| **Mã sinh viên:** | B23DCCN103 |
| **Lớp:** | D23CNPM03 |

**HÀ NỘI, THÁNG 10 NĂM 2026**

# LỜI CAM ĐOAN

Tôi xin cam đoan báo cáo tiểu luận môn học 'Phát triển Hệ thống Thông minh' là công trình nghiên cứu và thực nghiệm nghiêm túc được thực hiện bởi bản thân tôi dưới sự hướng dẫn khoa học của PGS. TS. Trần Đình Quế. Toàn bộ các kết quả khảo sát bộ dữ liệu (Dataset), mã nguồn thuật toán cài đặt từ đầu (Scratch Implementation trên nền tảng NumPy), mã nguồn thực thi trên các thư viện chuyên sâu (TensorFlow/Keras, PyTorch), các bảng biểu đối sánh số liệu và kiến trúc triển khai hệ thống (Deployment Pipeline) được trình bày trong báo cáo này đều được thực hiện dựa trên quy trình nghiên cứu khoa học trung thực, minh bạch và tuân thủ nghiêm ngặt các quy định về liêm chính học thuật.

Mọi thông tin trích dẫn, lý thuyết nền tảng, tập dữ liệu mở (Open-source Datasets từ Kaggle, UCI, Stanford, Zalando Research) đều có nguồn gốc rõ ràng và được ghi nhận đầy đủ trong phần Tài liệu tham khảo theo đúng chuẩn mực khoa học.

# LỜI CẢM ƠN

Lời đầu tiên, tôi xin bày tỏ lòng biết ơn sâu sắc nhất tới PGS. TS. Trần Đình Quế - Giảng viên phụ trách học phần 'Phát triển Hệ thống Thông minh' tại Học viện Công nghệ Bưu chính Viễn thông. Thầy đã truyền đạt những bài giảng sâu sắc về triết lý phát triển hệ thống thông minh ('Từ biểu diễn dữ liệu đến suy luận và hành động'), định hướng tư duy toán học giải tích đằng sau các thuật toán học máy, và rèn luyện cho sinh viên kỹ năng làm chủ công nghệ từ bản chất toán học (Scratch) đến các framework hiện đại (PyTorch, TensorFlow) và tư duy triển khai ứng dụng thực tế (Production Deployment).

Tôi cũng xin gửi lời cảm ơn chân thành đến Ban Giám đốc Học viện Công nghệ Bưu chính Viễn thông, các thầy cô giáo trong Khoa Công nghệ Thông tin đã tạo điều kiện tối đa về cơ sở vật chất, môi trường học thuật chất lượng cao và tài liệu nghiên cứu phong phú để tôi có thể hoàn thành tốt tiểu luận này.

# Mục lục

**LỜI CAM ĐOAN** ..... **2**
**LỜI CẢM ƠN** ..... **2**
**DANH MỤC TỪ VIẾT TẮT** ..... **5**
**MỞ ĐẦU** ..... **6**
1. Tính Cấp Thiết của Đề Tài ..... 6
2. Mục Tiêu Nghiên Cứu và Nhiệm Vụ Trọng Tâm ..... 6
3. Đối Tượng và Phạm Vi Nghiên Cứu ..... 7
4. Phương Pháp Luận và Cách Tiếp Cận Nghiên Cứu ..... 7
5. Ý Nghĩa Khoa Học và Giá Trị Thực Tiễn ..... 8
6. Bố Cục Báo Cáo Tiểu Luận ..... 8
**CHƯƠNG 1: LỊCH SỬ PHÁT TRIỂN TRÍ TUỆ NHÂN TẠO (AI)** ..... **9**
1.1 Khởi Nguyên Tư Duy Máy Móc và Giai Đoạn Tiền AI (Trước 1956) ..... 9
1.2 Hội Thảo Dartmouth (1956) và Thời Kỳ Hoàng Kim Đầu Tiên (1956 - 1974) ..... 9
1.3 Mùa Đông AI Thứ Nhất và Giới Hạn của Perceptron Đơn Lớp (1974 - 1980) ..... 10
1.4 Kỷ Nguyên Hệ Chuyên Gia và Mùa Đông AI Thứ Hai (1980 - 1993) ..... 11
1.5 Sự Trỗi Dậy của Học Máy Thống Kê và Dữ Liệu Lớn (1993 - 2011) ..... 12
1.6 Cuộc Cách Mạng Deep Learning và Điện Toán GPU (2012 - 2017) ..... 12
1.7 Kỷ Nguyên Transformer, Mô Hình Ngôn Ngữ Lớn và Generative AI (2017 - Nay) ..... 13
1.8 Khảo Sát 3 Bộ Dữ Liệu Lịch Sử Đặt Nền Móng cho AI ..... 13
1.9 Đánh Giá Tổng Kết và Bài Học Lịch Sử ..... 15
**CHƯƠNG 2: CÁC KỸ THUẬT MACHINE LEARNING CƠ BẢN** ..... **16**
2.1 Bản Chất Toán Học của Học Máy Có Giám Sát (Supervised Learning) ..... 16
2.2 Phân Tích Các Mô Hình Học Máy Cơ Bản ..... 17
2.3 Khảo Sát Chi Tiết 3 Bộ Dữ Liệu Thực Nghiệm cho Học Máy ..... 17
2.4 Cài Đặt Thực Nghiệm 3 Phiên Bản (Scratch - Keras - PyTorch) ..... 19
2.5 Đánh Giá Đối Sánh Hiệu Năng 3 Mô Hình ..... 21
2.6 Triển Khai Hệ Thống (Deployment Pipeline với FastAPI) ..... 22
**CHƯƠNG 3: MẠNG NƠ-RON TÍCH CHẬP (CONVOLUTIONAL NEURAL NETWORKS)** ..... **24**
3.1 Cơ Sở Toán Học của Phép Tích Chập và Trích Xuất Đặc Trưng Không Gian ..... 24
3.2 Khảo Sát Chi Tiết 3 Bộ Dữ Liệu Thị Giác Máy Tính ..... 26
3.3 Cài Đặt Toàn Diện 3 Phiên Bản Mô Hình CNN ..... 26
3.4 Đánh Giá Đối Sánh Thực Nghiệm và Phân Tích Sai Số ..... 29
3.5 Triển Khai Dịch Vụ Thị Giác Máy Tính Thực Tế với ONNX & REST API ..... 30

**CHƯƠNG 4: MẠNG NƠ-RON HỒI QUY (RECURRENT NEURAL NETWORKS)..... 31**
4.1 Bản Chất Dữ Liệu Chuỗi Thời Gian và Hạn Chế của Vanilla RNN..... 31
4.2 Kiến Trúc Giải Quyết Phụ Thuộc Dài Hạn: LSTM và GRU..... 32
4.3 Khảo Sát Chi Tiết 3 Bộ Dữ Liệu Chuỗi Thời Gian và Dữ Liệu Tuần Tự..... 33
4.4 Cài Đặt Toàn Diện 3 Phiên Bản Mô Hình Chuỗi (Scratch - Keras - PyTorch)..... 34
4.5 Thực Nghiệm Đánh Giá Đối Sánh Dự Báo Chuỗi Thời Gian Tài Chính..... 36
4.6 Triển Khai Dịch Vụ Dự Báo Chuỗi Thời Gian Thời Gian Thực..... 36
**TỔNG KẾT VÀ ĐỊNH HƯỚNG GIAI ĐOẠN TIẾP THEO..... 39**
1. Tổng Kết Kết Quả Đạt Được Giai Đoạn 1 (Mở đầu -> Chương 4)..... 39
2. Lộ Trình Phát Triển Hệ Thống Thông Minh Giai Đoạn 2 (Chương 5, 6, 7)..... 39
**TÀI LIỆU THAM KHẢO..... 40**

# DANH MỤC TỪ VIẾT TẮT

| Từ viết tắt | Thuật ngữ tiếng Anh | Ý nghĩa / Giải thích tiếng Việt |
|---|---|---|
| AI | Artificial Intelligence | Trí tuệ Nhân tạo |
| ML | Machine Learning | Học máy |
| DL | Deep Learning | Học sâu |
| ANN | Artificial Neural Network | Mạng Nơ-ron Nhân tạo |
| CNN | Convolutional Neural Network | Mạng Nơ-ron Tích chập |
| RNN | Recurrent Neural Network | Mạng Nơ-ron Hồi quy |
| LSTM | Long Short-Term Memory | Mạng Bộ nhớ Dài - Ngắn hạn |
| GRU | Gated Recurrent Unit | Đơn vị Hồi quy có Cổng |
| BPTT | Backpropagation Through Time | Lan truyền ngược qua Thời gian |
| BCE | Binary Cross-Entropy | Hàm mất mát Entropy chéo Nhị phân |
| MSE | Mean Squared Error | Sai số Toàn phương Trung bình |
| RMSE | Root Mean Squared Error | Căn bậc hai Sai số Toàn phương Trung bình |
| MAE | Mean Absolute Error | Sai số Tuyệt đối Trung bình |
| EDA | Exploratory Data Analysis | Phân tích Khám phá Dữ liệu |
| SVM | Support Vector Machine | Máy Vector Hỗ trợ |
| PCA | Principal Component Analysis | Phân tích Thành phần Chính |
| SE-Block | Squeeze-and-Excitation Block | Khối Chú ý Nén và Kích hoạt theo Kênh |
| ONNX | Open Neural Network Exchange | Định dạng Chuẩn trao đổi Mô hình Nơ-ron Mở |
| API | Application Programming Interface | Giao diện Lập trình Ứng dụng |
| MLOps | Machine Learning Operations | Vận hành và Triển khai Học máy Công nghiệp |
| LLM | Large Language Model | Mô hình Ngôn ngữ Lớn |
| RAG | Retrieval-Augmented Generation | Tạo sinh Tăng cường Truy xuất |
| KG | Knowledge Graph | Đồ thị Tri thức |

# MỞ ĐẦU

## 1. Tính Cấp Thiết của Đề Tài

Trong dòng chảy mạnh mẽ của cuộc Cách mạng Công nghiệp lần thứ tư, Trí tuệ Nhân tạo (Artificial Intelligence - AI) đã và đang khẳng định vị thế là công nghệ trụ cột, làm thay đổi căn bản diện mạo của mọi lĩnh vực từ sản xuất tự động, tài chính ngân hàng, chẩn đoán y tế, đến giao thông thông minh và giáo dục cá nhân hóa. Nếu như trong những thập niên cuối thế kỷ XX, các hệ thống thông minh chủ yếu dựa trên các tập luật tĩnh (Rule-based Systems) và tri thức chuyên gia thu hẹp, thì bước sang kỷ nguyên số hóa với sự bùng nổ của Dữ liệu lớn (Big Data) và năng lực tính toán vượt bậc của các bộ vi xử lý đồ họa (GPU/TPU), các mô hình dựa trên dữ liệu (Data-driven Models) - đặc biệt là Học máy (Machine Learning) và Học sâu (Deep Learning) - đã tạo nên những bước nhảy vọt mang tính cách mạng.

Tuy nhiên, một thực tế thách thức trong đào tạo kỹ sư công nghệ thông tin và khoa học dữ liệu hiện nay là hiện tượng 'hộp đen' (Black-box Syndrome): các nhà phát triển thường có xu hướng phụ thuộc hoàn toàn vào các thư viện bậc cao (như TensorFlow, PyTorch hay Scikit-learn), gọi các hàm có sẵn mà không thấu hiểu tường tận bản chất toán học, các phép biến đổi tensor, luồng lan truyền xuôi (Forward Propagation) và giải thuật lan truyền ngược (Backward Propagation / Autograd) diễn ra bên dưới. Điều này dẫn đến sự bế tắc khi mô hình gặp lỗi hội tụ, suy giảm độ dốc (Vanishing Gradient), quá khớp nghiêm trọng (Overfitting) hoặc khi cần tối ưu hóa tài nguyên phần cứng để triển khai trên các thiết bị biên (Edge Devices).

Bên cạnh đó, việc xây dựng một hệ thống thông minh hoàn chỉnh không chỉ dừng lại ở việc huấn luyện mô hình đạt chỉ số đo lường cao trong phòng thí nghiệm (Môi trường Jupyter Notebook tĩnh), mà mục tiêu tối hậu là phải hoàn thiện toàn bộ vòng đời phát triển hệ thống (System Lifecycle): từ khâu hiểu bản chất và biểu diễn dữ liệu (Data Representation), làm sạch và phân tích khám phá (EDA), huấn luyện mô hình toán học giải tích từ đầu (Scratch), đối sánh với các framework công nghiệp (Keras, PyTorch), và cuối cùng là đóng gói, tối ưu hóa suy luận và triển khai (Deployment) thành một dịch vụ Web/API sẵn sàng phục vụ người dùng cuối.

## 2. Mục Tiêu Nghiên Cứu và Nhiệm Vụ Trọng Tâm

Tiểu luận này hướng tới các mục tiêu nghiên cứu và nhiệm vụ trọng tâm sau:

- **Mục tiêu 1:** Hệ thống hóa toàn diện tiến trình lịch sử phát triển của AI: Làm rõ các cột mốc thăng trầm lịch sử từ thời kỳ phôi thai (1940s), hai Mùa đông AI, thời kỳ phục hưng của Machine Learning thống kê, cuộc cách mạng Deep Learning, cho đến kỷ nguyên Transformer và AI tạo sinh hiện đại.
- **Mục tiêu 2:** Làm chủ bản chất toán học và thuật toán qua 3 phương thức cài đặt: Đối với từng lớp bài toán (Machine Learning cơ bản, Mạng nơ-ron tích chập CNN, và Mạng nơ-ron hồi quy RNN), thực hiện cài đặt song song 3 phiên bản mã nguồn: (1) Cài đặt thuần từ đầu (Scratch Implementation bằng NumPy) để hiểu rõ từng phép đạo hàm giải tích; (2) Cài đặt trên

TensorFlow/Keras đại diện cho thiết kế trực quan công nghiệp; (3) Cài đặt trên PyTorch đại diện cho kiến trúc động linh hoạt chuẩn nghiên cứu khoa học.

- **Mục tiêu 3:** Khảo sát sâu sắc các bộ dữ liệu đa dạng (Multi-domain Datasets): Trong mỗi chương, tiến hành khảo sát từ 2 đến 3 bộ dữ liệu thực tế: cung cấp đường dẫn truy cập chuẩn, thông số kích thước (samples, features, dimensionality), trích xuất mẫu dữ liệu thực tế, phân tích phân bố thống kê (EDA), ma trận tương quan và nhận xét chuyên sâu về tính chất đại số của dữ liệu.
- **Mục tiêu 4:** Đánh giá thực nghiệm và phân tích đối sánh đa chiều: Đo lường, so sánh định lượng giữa 3 mô hình trên các tập dữ liệu dựa trên hệ thống chỉ số toàn diện (Accuracy, Precision, Recall, F1-Score, Loss, Training Time, Inference Latency, FLOPs), vẽ biểu đồ đường cong học tập và phân tích nguyên nhân sai số.
- **Mục tiêu 5:** Thiết kế kiến trúc triển khai sản phẩm thực tế (Deployment): Hiện thực hóa mô hình thành dịch vụ phần mềm có thể phục vụ trong thực tế thông qua việc đóng gói mô hình (Serialization / ONNX), xây dựng REST API thời gian thực với FastAPI/Flask và chuẩn hóa quy trình MLOps.

## 3. Đối Tượng và Phạm Vi Nghiên Cứu

Đối tượng nghiên cứu của tiểu luận bao gồm: các nguyên lý biểu diễn dữ liệu số học; các mô hình học máy kinh điển (Linear Regression, Logistic Regression, SVM, Random Forest); kiến trúc mạng nơ-ron tích chập (CNN) cùng các khối cải tiến hiện đại (Residual Connection, Squeeze-and-Excitation); và kiến trúc mạng nơ-ron hồi quy (Vanilla RNN, LSTM, GRU) cho chuỗi thời gian.

Phạm vi nghiên cứu giai đoạn 1 (Tuần báo cáo hiện tại) tập trung hoàn thiện toàn diện từ Phần Mở đầu đến hết Chương 4, tạo tiền đề vững chắc để tiếp tục mở rộng sang các công nghệ tối tân ở giai đoạn 2 gồm: Đồ thị tri thức & RAG (Chương 5), Mạng nơ-ron đồ thị sâu Deep Graph (Chương 6), và Hệ đa tác tử thông minh Multi-Agent Systems (Chương 7).

## 4. Phương Pháp Luận và Cách Tiếp Cận Nghiên Cứu

Tiểu luận áp dụng phương pháp luận kết hợp chặt chẽ giữa nghiên cứu lý thuyết giải tích và thực nghiệm khoa học dựa trên số liệu thực tế:

- **1. Lý thuyết:** Phương pháp giải tích toán học: Thiết lập các bài toán tối ưu dưới dạng hàm mục tiêu (Objective Function), khai triển đạo hàm riêng giải tích theo từng ma trận trọng số và vector độ lệch thông qua quy tắc chuỗi (Chain Rule).
- **2. Thực nghiệm:** Phương pháp lập trình thực nghiệm đối chứng: Thiết lập môi trường tính toán đồng nhất (cùng random seed, cùng tỷ lệ chia tập Train/Validation/Test, cùng cấu hình tiền xử lý chuẩn hóa dữ liệu Standard Scaler) để đảm bảo tính tái lập (Reproducibility) và tính công bằng tuyệt đối giữa 3 phương thức cài đặt (Scratch vs Keras vs PyTorch).
- **3. Triển khai:** Phương pháp kỹ nghệ phần mềm và MLOps: Áp dụng kiến trúc microservices phân tách giữa tầng tiền xử lý dữ liệu, tầng mô hình tính toán và tầng giao diện lập trình ứng dụng RESTful API.

## 5. Ý Nghĩa Khoa Học và Giá Trị Thực Tiễn

Về mặt khoa học: Báo cáo cung cấp một tài liệu tham khảo hệ thống, chuẩn mực và chi tiết, kết nối một cách liền mạch giữa lý thuyết giải tích toán học trừu tượng với các dòng mã nguồn thực thi cụ thể, giúp xóa bỏ hoàn toàn tính chất 'hộp đen' của các mô hình học sâu.

Về mặt thực tiễn: Toàn bộ các bộ dữ liệu khảo sát đều lấy từ các nguồn thực tế uy tín thế giới (chẩn đoán y sinh học, dự báo giá bất động sản, thị giác nhận dạng vật thể, và chuỗi tài chính giao dịch vàng/bạc quốc tế). Các mã nguồn triển khai REST API cung cấp một khuôn mẫu (Template) hoàn chỉnh sẵn sàng cho việc đóng gói vào các giải pháp phần mềm doanh nghiệp.

## 6. Bố Cục Báo Cáo Tiểu Luận

Báo cáo tiểu luận được tổ chức gồm 4 chương nội dung cốt lõi và phần tổng kết định hướng tương lai:

- **Phần Mở đầu:** Mở đầu: Đặt vấn đề, tính cấp thiết, mục tiêu và phương pháp nghiên cứu.
- **Chương 1:** Chương 1: Lịch sử phát triển Trí tuệ Nhân tạo - Toàn cảnh các giai đoạn thăng trầm, các bước chuyển mình mô thức (Paradigm Shifts) và 3 bộ dữ liệu kinh điển đặt nền móng AI.
- **Chương 2:** Chương 2: Các kỹ thuật Machine Learning cơ bản - Cơ sở toán học, khảo sát 3 bộ dữ liệu tabular (Diabetes, Housing, Wine), cài đặt chi tiết 3 phiên bản Scratch-Keras-PyTorch, thực nghiệm so sánh và triển khai REST API.
- **Chương 3:** Chương 3: Mạng nơ-ron Tích chập (CNN) - Toán học tích chập, Pooling, ResNet, SE-Block, khảo sát 3 bộ dữ liệu thị giác (MNIST, Fashion-MNIST, CIFAR-10), cài đặt 3 phiên bản, ma trận nhầm lẫn và triển khai ONNX API.
- **Chương 4:** Chương 4: Mạng nơ-ron Hồi quy (RNN) - Bản chất chuỗi, Vanilla RNN, BPTT, bài toán triệt tiêu gradient, LSTM, GRU, khảo sát 3 bộ dữ liệu chuỗi (Gold Prices, Jena Climate, IMDb), cài đặt 3 phiên bản, dự báo đa bước và dashboard triển khai.
- **Phần Kết luận:** Tổng kết & Định hướng: Đánh giá kết quả và lộ trình mở rộng sang Chương 5 (Knowledge Graph & RAG), Chương 6 (Deep Graph) và Chương 7 (Multi-Agent Systems).

# CHƯƠNG 1: LỊCH SỬ PHÁT TRIỂN TRÍ TUỆ NHÂN TẠO (AI)

## 1.1 Khởi Nguyên Tư Duy Máy Móc và Giai Đoạn Tiền AI (Trước 1956)

Khát vọng tạo ra những cỗ máy có khả năng tư duy và hành động thông minh như con người đã xuất hiện từ rất sớm trong lịch sử nhân loại, khởi nguồn từ các huyền thoại cổ đại về người máy Talos bằng đồng cho đến các phát kiến triết học duy lý của René Descartes và Thomas Hobbes ('Suy nghĩ chính là tính toán'). Tuy nhiên, chỉ đến giữa thế kỷ XX, khi nền tảng của lý thuyết tính toán (Computability Theory) và điều khiển học (Cybernetics) được xác lập, Trí tuệ Nhân tạo mới chính thức bước vào vũ đài khoa học thực nghiệm.

Cột mốc đầu tiên mang tính nền móng là công trình của Warren McCulloch và Walter Pitts vào năm 1943 với bài báo 'A Logical Calculus of the Ideas Immanent in Nervous Activity'. Lần đầu tiên, hai nhà khoa học đã trừu tượng hóa tế bào thần kinh sinh học thành một mô hình tính toán nhị phân logic đơn giản (mô hình nơ-ron McCulloch-Pitts). Mô hình này chỉ ra rằng sự kết hợp của các nơ-ron đơn giản có thể thực hiện mọi phép toán logic cơ bản (AND, OR, NOT), chứng minh rằng mạng lưới thần kinh nhân tạo có thể giải quyết các phép tính hình thức.

Năm 1950, nhà toán học thiên tài Alan Turing đã xuất bản bài báo kinh điển 'Computing Machinery and Intelligence' trên tạp chí Mind. Turing mở đầu bằng câu hỏi lịch sử: 'Máy móc có thể tư duy được không?' (Can machines think?). Để trả lời câu hỏi này một cách khách quan thay vì sa đà vào các định nghĩa triết học mơ hồ về ý thức, ông đã đề xuất 'Trò chơi Bắt chước' (The Imitation Game), ngày nay được biết đến với tên gọi Phép thử Turing (Turing Test). Nếu một máy tính có thể trò chuyện bằng văn bản với một giám khảo con người mà giám khảo đó không thể phân biệt được đối tượng trò chuyện là người hay máy với xác suất vượt quá 50%, thì cỗ máy đó được coi là có trí thông minh. Công trình của Turing đã đặt viên gạch đầu tiên định hình toàn bộ tư duy lý thuyết về trí tuệ nhân tạo hiện đại.

## 1.2 Hội Thảo Dartmouth (1956) và Thời Kỳ Hoàng Kim Đầu Tiên (1956 - 1974)

Mùa hè năm 1956, tại trường Đại học Dartmouth (Hanover, New Hampshire, Hoa Kỳ), một hội thảo học thuật lịch sử kéo dài hai tháng đã được tổ chức dưới sự chủ trì của John McCarthy, Marvin Minsky, Nathaniel Rochester và Claude Shannon. Chính tại sự kiện này, John McCarthy đã chính thức đề xuất và khẳng định thuật ngữ 'Trí tuệ Nhân tạo' (Artificial Intelligence - AI). Lời tuyên ngôn của hội thảo nêu rõ giả thuyết nền tảng: 'Mọi khía cạnh của việc học hay bất kỳ đặc tính nào khác của trí thông minh về mặt nguyên lý đều có thể được mô tả chính xác đến mức một cỗ máy có thể được chế tạo để mô phỏng nó'.

Hội thảo Dartmouth đã khai sinh ra một làn sóng nghiên cứu đầy hưng phấn, mở đầu cho Thời kỳ Hoàng kim Thứ nhất (The First Golden Age of AI). Trong giai đoạn này, các nhà khoa học tin rằng chỉ trong vòng 10 đến 20 năm, một cỗ máy có trí thông minh ngang ngửa con người (AGI) sẽ được tạo ra. Những thành tựu nổi bật bao gồm:

- **Hệ thống suy luận ký hiệu:** Logic Theorist (1956) và General Problem Solver (GPS, 1957) của Allen Newell, Cliff Shaw và Herbert Simon: Phần mềm chứng minh được 38 trên tổng số 52

định lý trong cuốn sách toán học kinh điển 'Principia Mathematica' của Russell và Whitehead, đặt nền móng cho trường phái AI biểu tượng (Symbolic AI).

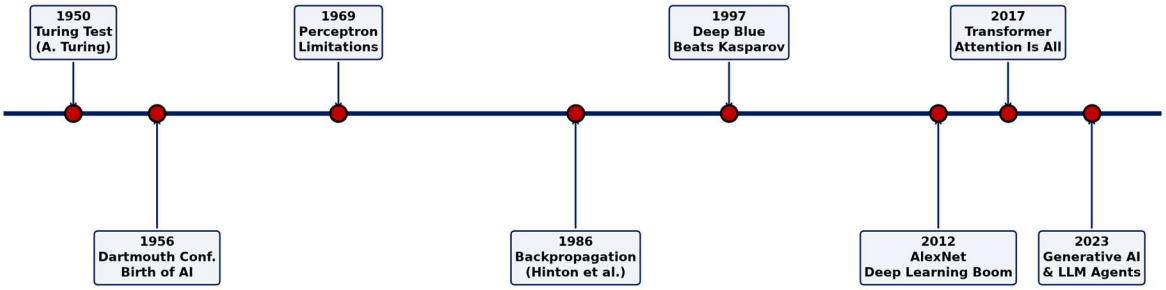
- **Học máy sơ khai:** Chương trình chơi cờ Đam của Arthur Samuel (1959): Hoạt động trên máy tính IBM 704, có khả năng học từ các ván cờ trước đó để tự nâng cao trình độ và đánh bại các kỳ thủ con người nghiệp dư. Samuel cũng là người đầu tiên đặt ra thuật ngữ 'Machine Learning' (Học máy).
- **Mạng nơ-ron nhân tạo:** Mô hình Perceptron của Frank Rosenblatt (1958): Triển khai trên máy tính Mark I Perceptron tại Phòng thí nghiệm Hàng không Cornell. Perceptron có khả năng học trọng số thông qua dữ liệu huấn luyện để phân loại các dạng hình học 20x20 pixel, trở thành mô hình mạng nơ-ron học máy có giám sát đầu tiên trên thế giới.
- **Xử lý ngôn ngữ tự nhiên:** Hệ thống ELIZA (1966) của Joseph Weizenbaum tại MIT: Một trong những chương trình xử lý ngôn ngữ tự nhiên đầu tiên mô phỏng một chuyên gia tâm lý học trị liệu, sử dụng kỹ thuật đối khớp mẫu (Pattern Matching).

### 1.3 Mùa Đông AI Thứ Nhất và Giới Hạn của Perceptron Đơn Lớp (1974 - 1980)

Sự lạc quan tột độ đã nhường chỗ cho thất vọng sâu sắc khi các nghiên cứu vấp phải những rào cản kỹ thuật không thể vượt qua vào thời điểm đó. Năm 1969, cuốn sách mang tính bước ngoặt 'Perceptrons' của Marvin Minsky và Seymour Papert được xuất bản. Bằng lập luận toán học chặt chẽ, hai tác giả đã chứng minh rằng mô hình Perceptron một lớp không thể giải quyết được các bài toán phân tách phi tuyến tính đơn giản, điển hình là hàm logic XOR (Exclusive OR). Hơn thế nữa, cuốn sách còn bày tỏ sự bi quan về khả năng mở rộng thuật toán học cho các mạng nơ-ron nhiều lớp (Multilayer Perceptrons).

Sự phê phán của Minsky và Papert đã giáng một đòn chí mạng vào trường phái kết nối (Connectionism / Neural Networks), khiến hầu hết các nguồn kinh phí tài trợ cho nghiên cứu mạng nơ-ron bị đình chỉ ngay lập tức. Cùng lúc đó, tại Vương quốc Anh, Báo cáo Lighthill (Lighthill Report, 1973) do Giáo sư James Lighthill trình lên Hội đồng Nghiên cứu Khoa học đã chỉ trích gay gắt sự thất bại của AI trong việc giải quyết các bài toán thế giới thực do hiện tượng 'bùng nổ tổ hợp' (Combinatorial Explosion). Tại Hoa Kỳ, Cơ quan Chỉ đạo các Dự án Nghiên cứu Tiên tiến Quốc phòng (DARPA) đã cắt giảm ngân sách quy mô lớn. Giai đoạn từ năm 1974 đến 1980 chính thức đi vào lịch sử với tên gọi 'Mùa đông AI Thứ nhất' (The First AI Winter), một giai đoạn đóng băng nghiên cứu và cạn kiệt nguồn tài trợ.

Hình 1.1: Sơ đồ các cột mốc lịch sử then chốt trong quá trình phát triển Trí tuệ Nhân tạo




***Hình 1.1: Trục thời gian tiến trình phát triển Trí tuệ Nhân tạo từ khởi nguyên đến kỷ nguyên Generative AI***

## 1.4 Kỷ Nguyên Hệ Chuyên Gia và Mùa Đông AI Thứ Hai (1980 - 1993)

Vào đầu những năm 1980, AI hồi sinh ngoạn mục nhờ sự xuất hiện của mô thức 'Hệ chuyên gia' (Expert Systems) thuộc trường phái AI biểu tượng. Thay vì cố gắng tìm kiếm các giải thuật giải quyết vấn đề tổng quát, các nhà khoa học tập trung vào việc mô hình hóa tri thức chuyên sâu của các chuyên gia con người trong từng lĩnh vực hẹp dưới dạng các tập luật 'Nếu - Thì' (IF-THEN Rules) và một động cơ suy luận (Inference Engine).

Các hệ thống tiêu biểu như DENDRAL (phân tích cấu trúc phân tử hóa học), MYCIN (chẩn đoán bệnh nhiễm trùng máu với độ chính xác tương đương các bác sĩ giỏi), và đặc biệt là hệ thống XCON (R1) của Tập đoàn Digital Equipment Corporation (DEC) giúp cấu hình tự động hệ thống máy tính VAX, tiết kiệm cho công ty hàng chục triệu USD mỗi năm. Sự thành công thương mại của Hệ chuyên gia đã kéo theo sự bùng nổ của thị trường phần cứng chuyên dụng chạy ngôn ngữ LISP (LISP Machines của Symbolics, LMI).

Cũng trong giai đoạn này, mạng nơ-ron nhân tạo đã có màn tái xuất lịch sử. Năm 1986, David Rumelhart, Geoffrey Hinton và Ronald Williams đã công bố bài báo kinh điển 'Learning representations by back-propagating errors' trên tạp chí Nature. Bài báo đã phổ biến giải thuật Lan truyền ngược (Backpropagation), giải quyết triệt để vấn đề huấn luyện trọng số cho mạng nơ-ron nhiều tầng, phá vỡ định kiến kéo dài gần hai thập kỷ của Minsky & Papert.

Tuy nhiên, 'bong bóng' Hệ chuyên gia nhanh chóng sụp đổ vào cuối những năm 1980. Các hệ chuyên gia bộc lộ điểm yếu chết người: tri thức quá cứng nhắc, chi phí duy trì và cập nhật luật cực kỳ tốn kém (vấn đề Knowledge Acquisition Bottleneck), không có khả năng tự học hỏi khi dữ liệu thay đổi và hoàn toàn tê liệt khi gặp các tình huống ngoại lệ. Cùng lúc đó, các máy tính để bàn cá nhân đa năng (PC của IBM và trạm máy tính của Sun Microsystems) với kiến trúc x86 trở nên mạnh mẽ hơn nhiều với chi phí rẻ hơn hàng chục lần so với máy tính LISP chuyên dụng. Toàn bộ ngành công nghiệp phần cứng LISP phá sản, kéo theo sự cắt giảm tài trợ từ DARPA và mở ra 'Mùa đông AI Thứ hai' (The Second AI Winter, 1987 - 1993).

## 1.5 Sự Trỗi Dậy của Học Máy Thống Kê và Dữ Liệu Lớn (1993 - 2011)

Rút bài học sâu sắc từ sự thất bại của các hệ thống luật cứng nhắc, từ giữa thập niên 1990, cộng đồng AI đã có một bước chuyển dịch mô thức căn bản (Paradigm Shift): từ việc lập trình tri thức thủ công sang việc tự động trích xuất quy luật từ dữ liệu thông qua lý thuyết xác suất thống kê (Statistical Machine Learning). AI chuyển hướng tiếp cận thực dụng, gắn chặt với toán học tối ưu, đại số tuyến tính và thống kê thực nghiệm.

Những bước đột phá lớn trong thời kỳ này bao gồm:

- **Mô hình biên cực đại:** Máy Vector Hỗ trợ (Support Vector Machines - SVM) do Vladimir Vapnik và Alexey Chervonenkis hoàn thiện (1995): Dựa trên lý thuyết học thống kê (Statistical Learning Theory) và nguyên lý Cực tiểu hóa Rủi ro Cấu trúc (Structural Risk Minimization). Với kỹ thuật 'Kernel Trick' khéo léo, SVM đã thống trị hoàn toàn các bài toán phân loại và hồi quy nhờ tính toán tối ưu lồi toàn cục và khả năng tổng quát hóa vượt trội.
- **Mô hình hợp quần (Ensemble Learning):** Rừng ngẫu nhiên (Random Forest) do Leo Breiman đề xuất (2001): Kết hợp kỹ thuật lấy mẫu tái lặp Bagging và lựa chọn ngẫu nhiên tập đặc trưng, trở thành thuật toán ensemble mạnh mẽ hàng đầu cho dữ liệu dạng bảng.
- **Cột mốc trò chơi trí tuệ:** Cột mốc Deep Blue (1997): Siêu máy tính của IBM đánh bại nhà vô địch cờ vua thế giới Garry Kasparov trong một trận đấu tiêu chuẩn 6 ván, khẳng định sức mạnh của thuật toán tìm kiếm cây trò chơi minimax kết hợp tăng tốc phần cứng.
- **Hạ tầng dữ liệu:** Sự bùng nổ của Internet và Dữ liệu lớn (Big Data): Các công cụ tìm kiếm (Google PageRank), mạng xã hội và thương mại điện tử đã tích lũy hàng petabyte dữ liệu, tạo ra mảnh đất màu mỡ cho các mô hình học máy phát huy tối đa năng lực.

## 1.6 Cuộc Cách Mạng Deep Learning và Điện Toán GPU (2012 - 2017)

Tháng 10 năm 2012, tại Hội nghị NIPS danh giá, Alex Krizhevsky, Ilya Sutskever và Geoffrey Hinton đã giới thiệu mô hình mạng nơ-ron tích chập AlexNet tham gia cuộc thi nhận dạng hình ảnh ImageNet Large Scale Visual Recognition Challenge (ILSVRC). AlexNet đã giành chiến thắng áp đảo khi giảm tỷ lệ lỗi Top-5 từ 26.2% xuống còn 15.3% (bỏ xa đội về nhì hơn 10%), tạo nên một cơn địa chấn học thuật toàn cầu.

Thành công của AlexNet là sự hội tụ hoàn hảo của 3 nhân tố cốt lõi: (1) Bộ dữ liệu khổng lồ ImageNet với 1.2 triệu ảnh có gán nhãn do Fei-Fei Li xây dựng; (2) Năng lực tính toán song song mạnh mẽ của bộ vi xử lý đồ họa GPU NVIDIA GeForce GTX 580; và (3) Các kỹ thuật thuật toán tiên tiến như hàm kích hoạt ReLU giúp triệt tiêu hiện tượng bão hòa đạo hàm, lớp Dropout chống quá khớp, và Data Augmentation mở rộng không gian mẫu.

Sự kiện AlexNet đã mở đầu cho kỷ nguyên bùng nổ của Học sâu (Deep Learning Boom). Trong những năm tiếp theo, hàng loạt kiến trúc mạng nơ-ron sâu mang tính đột phá liên tục ra đời: VGGNet (2014) chuẩn hóa kích thước kernel tích chập nhỏ 3x3; GoogLeNet / Inception (2014) tích hợp đa tỷ lệ không gian; ResNet (He et al., 2015) đưa vào kết nối tắt phần dư (Skip/Residual Connections) cho phép huấn luyện các mạng cực sâu lên tới 152 tầng mà không lo suy giảm gradient, chính thức vượt qua độ chính xác của thị giác con người trên ImageNet.

Năm 2016, hệ thống AlphaGo của Google DeepMind do Demis Hassabis lãnh đạo đã đánh bại huyền thoại cờ vây Lee Sedol với tỷ số 4-1. Cờ vây vốn được xem là đỉnh cao trí tuệ con người với không gian trạng thái $10^{170}$(lớn hơn số nguyên tử trong vũ trụ quan sát được). AlphaGo đã chứng minh sức mạnh kết hợp tuyệt đỉnh giữa Mạng nơ-ron tích chập sâu, Tìm kiếm cây Monte Carlo (MCTS) và Học tăng cường sâu (Deep Reinforcement Learning).

## 1.7 Kỷ Nguyên Transformer, Mô Hình Ngôn Ngữ Lớn và Generative AI (2017 - Nay)

Năm 2017, các nhà nghiên cứu tại Google công bố bài báo lịch sử mang tựa đề 'Attention Is All You Need', giới thiệu kiến trúc Transformer. Bằng việc loại bỏ hoàn toàn các cấu trúc hồi quy tuần tự của RNN/LSTM và thay thế bằng cơ chế Tự chú ý Đa đầu (Multi-Head Self-Attention), Transformer cho phép xử lý song song toàn bộ chuỗi văn bản trên quy mô siêu máy tính.

Kiến trúc Transformer đã trở thành nền tảng thống trị toàn bộ thế giới AI hiện đại: từ các mô hình ngôn ngữ hiểu theo ngữ cảnh như BERT (2018), cho đến dòng mô hình tự hồi quy tạo sinh khổng lồ GPT-2, GPT-3, GPT-4 của OpenAI; các mô hình mã nguồn mở LLaMA của Meta; và các mô hình đa phương thức (Multimodal AI) xử lý đồng thời văn bản, âm thanh, hình ảnh và video. AI đã chuyển mình từ giai đoạn nhận dạng (Perception) và phân loại tĩnh sang giai đoạn Tạo sinh (Generative AI) và Hành động tự chủ thông qua các Hệ đa tác tử (Multi-Agent Systems).

## 1.8 Khảo Sát 3 Bộ Dữ Liệu Lịch Sử Đặt Nền Móng cho AI

Để hiểu sâu sắc sự tiến hóa của AI từ thuở sơ khai, tiểu luận tiến hành khảo sát chi tiết 3 bộ dữ liệu mang tính biểu tượng lịch sử đã đặt nền móng cho các thuật toán nhận dạng mẫu, chẩn đoán y sinh và hồi quy thống kê:

### 1.8.1 Bộ Dữ Liệu Iris Flower (Fisher, 1936)

- **Nguồn dữ liệu:** Đường dẫn truy cập chính thức (Link): https://archive.ics.uci.edu/dataset/53/iris
- **Thông số kỹ thuật:** Kích thước & Cấu trúc (Size & Shape): 150 mẫu (instances), 4 đặc trưng số thực (features), 1 cột nhãn phân loại. Dung lượng file: ~4.5 KB.
- **Trích xuất mẫu:** Bảng mẫu dữ liệu đại diện trích xuất từ tập dữ liệu:

**Bảng: Trích xuất 5 mẫu dữ liệu đại diện từ Tập dữ liệu Iris (Fisher, 1936)**

| ID | Sepal Length (cm) | Sepal Width (cm) | Petal Length (cm) | Petal Width (cm) | Species (Loài hoa) |
|---|---|---|---|---|---|
| 1 | 5.1 | 3.5 | 1.4 | 0.2 | Iris-setosa |
| 2 | 4.9 | 3.0 | 1.4 | 0.2 | Iris-setosa |
| 51 | 7.0 | 3.2 | 4.7 | 1.4 | Iris-versicolor |
| 52 | 6.4 | 3.2 | 4.5 | 1.5 | Iris-versicolor |
| 101 | 6.3 | 3.3 | 6.0 | 2.5 | Iris-virginica |

- **Nhận xét EDA:** Phân bố thống kê & Nhận xét học thuật: Tập dữ liệu phân bố cân bằng tuyệt đối giữa 3 lớp (50 mẫu Setosa, 50 mẫu Versicolor, 50 mẫu Virginica). Đặc trưng Petal Length và Petal Width có khả năng phân tách cực kỳ mạnh mẽ: lớp Setosa hoàn toàn phân tách tuyến tính

(Linearly Separable) khỏi hai lớp còn lại trong không gian 2D. Trong khi đó, Versicolor và Virginica có vùng giao thoa phi tuyến nhẹ, trở thành chuẩn mực kiểm thử kinh điển cho các giải thuật phân loại tuyến tính (LDA, Perceptron) và phi tuyến (SVM, MLP).

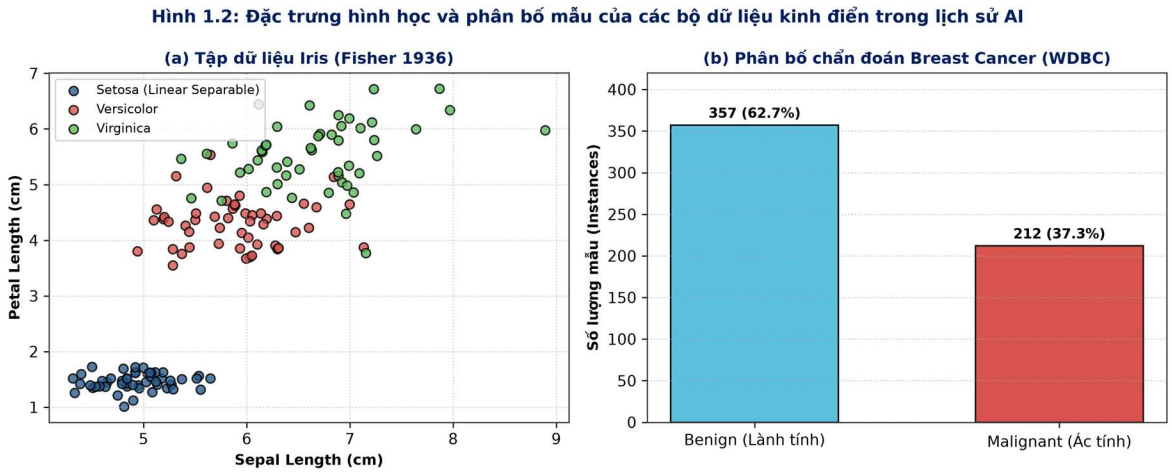
### 1.8.2 Bộ Dữ Liệu Breast Cancer Wisconsin Diagnostic (WDBC)

- **Nguồn dữ liệu:** Đường dẫn truy cập chính thức (Link): https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic
- **Thông số kỹ thuật:** Kích thước & Cấu trúc: 569 mẫu bệnh nhân, 30 đặc trưng số thực đo lường từ hình ảnh tế bào chọc hút kim nhỏ (FNA). Dung lượng: ~124 KB. Nhãn: Benign (Lành tính - B) hoặc Malignant (Ác tính - M).
- **Trích xuất mẫu:** Bảng mẫu dữ liệu đại diện:

**Bảng: Trích xuất 5 mẫu đại diện từ Breast Cancer Wisconsin Diagnostic Dataset**

| ID | Radius Mean | Texture Mean | Perimeter Mean | Area Mean | Smoothness | Diagnosis |
|---|---|---|---|---|---|---|
| 842302 | 17.99 | 10.38 | 122.80 | 1001.0 | 0.1184 | M (Ác tính) |
| 842517 | 20.57 | 17.77 | 132.90 | 1326.0 | 0.0847 | M (Ác tính) |
| 84300903 | 19.69 | 21.25 | 130.00 | 1203.0 | 0.1096 | M (Ác tính) |
| 8510426 | 13.54 | 14.36 | 87.46 | 566.3 | 0.0977 | B (Lành tính) |
| 8510653 | 13.08 | 15.71 | 85.63 | 520.0 | 0.1075 | B (Lành tính) |

- **Nhận xét EDA:** Phân bố & Nhận xét: Dữ liệu gồm 357 mẫu lành tính (62.7%) và 212 mẫu ác tính (37.3%). Các thuộc tính hình học tế bào (Radius, Perimeter, Area) có hệ số tương quan Pearson rất cao (> 0.98), thể hiện hiện tượng đa cộng tuyến (Multicollinearity). Tập dữ liệu minh chứng cho tầm quan trọng của việc chuẩn hóa thang đo (Standard Scaling) và giảm chiều dữ liệu (PCA) trong các ứng dụng chẩn đoán y tế cổ điển.




*Hình 1.2: Không gian đặc trưng hình học và phân bố nhãn của các bộ dữ liệu kinh điển trong lịch sử AI*

### 1.8.3 Bộ Dữ Liệu California Housing Dataset (Pace & Barry, 1997)

- **Nguồn dữ liệu:** Đường dẫn truy cập chính thức: https://www.kaggle.com/datasets/camnugent/california-housing-prices
- **Thông số kỹ thuật:** Kích thước & Cấu trúc: 20,640 mẫu khối điều tra dân số (census block groups), 8 đặc trưng đầu vào kinh tế - địa lý và 1 biến mục tiêu liên tục (MedHouseVal). Dung lượng: ~1.4 MB.
- **Nhận xét EDA:** Phân bố & Nhận xét: Biến mục tiêu Median House Value bị chặn trên (Capped) ở ngưỡng $500,000, tạo ra một đỉnh bất thường tại cực phải của histogram. Thuộc tính thu nhập trung bình (MedInc) có hệ số tương quan dương mạnh nhất với giá nhà ($r$ xấp xỉ 0.68$). Đây là bộ dữ liệu chuẩn mực hàng đầu để đánh giá các giải thuật hồi quy tuyến tính, cây quyết định và mạng nơ-ron nhiều lớp.

## 1.9 Đánh Giá Tổng Kết và Bài Học Lịch Sử

Lịch sử hơn 70 năm của Trí tuệ Nhân tạo để lại cho các kỹ sư phát triển hệ thống thông minh những bài học sâu sắc:

1. Không có giải pháp 'viên đạn bạc' (No Silver Bullet): Mọi mô hình toán học đều có những giới hạn nội tại gắn liền với không gian giả thuyết của nó. Việc nắm vững cả AI biểu tượng, Machine Learning thống kê và Deep Learning là điều kiện tiên quyết để xây dựng các hệ thống lai (Neuro-Symbolic AI) vững chắc.

2. Dữ liệu là trọng tâm cốt lõi (Data-Centric AI): Thuật toán mạnh mẽ nhất cũng trở nên vô dụng nếu dữ liệu đầu vào bị sai lệch, thiếu chuẩn hóa hoặc không đại diện cho thế giới thực. Hiểu sâu cấu trúc đại số và phân bố của dữ liệu luôn là bước đi tiên quyết trước khi viết bất kỳ dòng mã huấn luyện nào.

# CHƯƠNG 2: CÁC KỸ THUẬT MACHINE LEARNING CƠ BẢN

## 2.1 Bản Chất Toán Học của Học Máy Có Giám Sát (Supervised Learning)

Học máy có giám sát là quá trình tìm kiếm một hàm ánh xạ $f: \mathcal{X} \to \mathcal{Y}$ từ không gian đặc trưng đầu vào $\mathcal{X} \subseteq \mathbb{R}^D$ sang không gian nhãn đầu ra $\mathcal{Y}$, dựa trên tập dữ liệu huấn luyện gồm $N$ cặp quan sát đã biết trước nhãn: $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$, trong đó mỗi vector đặc trưng $x^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_D^{(i)}\right]^T \in \mathbb{R}^D$.

Dưới góc nhìn tối ưu hóa toán học, quá trình học thực chất là việc tìm kiếm vector tham số tối ưu $heta^* = [W^*, b^*]$ trong không gian giả thuyết $\mathcal{H}$ sao cho cực tiểu hóa hàm mất mát thực nghiệm (Empirical Risk Minimization - ERM) kết hợp với thành phần điều chuẩn chống quá khớp (Regularization):

$$\theta^* = \arg\min_{\theta} \left\{ \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\left(f\left(x^{(i)}; \theta\right), y^{(i)}\right) + \lambda \mathcal{R}(\theta) \right\} \quad \textbf{(2.1)}$$

Trong đó $\mathcal{L}(\hat{y}, y)$ là hàm mất mát đo lường độ sai lệch giữa dự đoán $\hat{y}$ và giá trị thực tế $y$; $\mathcal{R}(heta)$ là hàm phạt điều chuẩn ($L_2$ Regularization: $\frac{1}{2}\|W\|_2^2$ hoặc $L_1$ Regularization: $\|W\|_1$); và $\lambda > 0$ là siêu tham số kiểm soát mức độ phạt.

### 2.1.1 Các Hàm Mất Mát Cốt Lõi

- **1. MSE Loss:** Hàm Mất Mát Toàn Phương Trung Bình (Mean Squared Error - MSE) dùng cho bài toán Hồi quy (Regression):

$$\mathcal{L}_{\text{MSE}}(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2 \quad \Longrightarrow \quad \mathcal{J}_{\text{MSE}}(W, b) = \frac{1}{2N} \sum_{i=1}^{N} \left(y^{(i)} - \left(W^T x^{(i)} + b\right)\right)^2 \quad \textbf{(2.2)}$$

- **2. Binary Cross-Entropy:** Hàm Mất Mát Entropy Chéo Nhị Phân (Binary Cross-Entropy - BCE) dùng cho bài toán Phân loại 2 lớp (Binary Classification):

$$\mathcal{L}_{\text{BCE}}(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})] \quad \textbf{(2.3)}$$

Với $\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$ là xác suất hậu nghiệm đầu ra của hàm kích hoạt Sigmoid với $z = W^T x + b$.

### 2.1.2 Giải Thuật Tối Ưu Gradient Descent và Đạo Hàm Giải Tích

Để cực tiểu hóa hàm chi phí $\mathcal{J}(W, b)$, giải thuật Gradient Descent cập nhật tham số ngược chiều với vector đạo hàm riêng tại mỗi bước lặp với tốc độ học (learning rate) $\eta > 0$:

$$W^{(t+1)} = W^{(t)} - \eta \frac{\partial \mathcal{J}}{\partial W}, \qquad b^{(t+1)} = b^{(t)} - \eta \frac{\partial \mathcal{J}}{\partial b} \tag{2.4}$$

Áp dụng quy tắc chuỗi giải tích (Chain Rule) cho mô hình Logistic Regression với ma trận hóa toàn phần trên tập dữ liệu $X \in \mathbb{R}^{N \times D}$, $y \in \mathbb{R}^{N \times 1}$:

$$\frac{\partial \mathcal{J}}{\partial z} = \hat{y} - y \quad \Longrightarrow \quad \frac{\partial \mathcal{J}}{\partial W} = \frac{1}{N} X^T (\hat{y} - y), \qquad \frac{\partial \mathcal{J}}{\partial b} = \frac{1}{N} \sum_{i=1}^{N} \left( \hat{y}^{(i)} - y^{(i)} \right) \tag{2.5}$$

Đây chính là biểu thức đạo hàm đẹp đẽ và then chốt nhất giúp chúng ta có thể tự cài đặt thuật toán từ đầu (Scratch Implementation) bằng các phép nhân ma trận thuần túy trong thư viện NumPy mà không cần bất kỳ công cụ tự động vi phân nào.

## 2.2 Phân Tích Các Mô Hình Học Máy Cơ Bản

1. Logistic Regression: Mô hình tuyến tính tổng quát sử dụng hàm liên kết logit. Ưu điểm nổi bật là tốc độ tính toán cực nhanh, chi phí bộ nhớ thấp và kết quả đầu ra có ý nghĩa xác suất rõ ràng ($P(Y = 1|X)$). Tuy nhiên, mô hình có đường biên quyết định tuyến tính (Hyperplane) nên hạn chế đối với dữ liệu phân tách phức tạp.

2. Máy Vector Hỗ trợ (Support Vector Machine - SVM): Tìm kiếm siêu phẳng phân tách tối ưu hóa độ rộng của lề (Margin). Điểm mạnh vượt trội của SVM là sự phụ thuộc hoàn toàn vào tập các vector hỗ trợ (Support Vectors) nằm sát biên giới, kết hợp với các hàm nhân (RBF / Polynomial Kernel) giúp giải quyết triệt để các bài toán phi tuyến trong không gian vô hạn chiều.

3. Cây Quyết định (Decision Tree) & Rừng Ngẫu nhiên (Random Forest): Phân chia không gian dữ liệu bằng các siêu phẳng trực giao song song với các trục tọa độ dựa trên độ giảm chỉ số vẩn đục Gini hoặc độ lợi thông tin (Information Gain / Shannon Entropy). Rừng ngẫu nhiên khử bỏ triệt để hiện tượng quá khớp của cây đơn lẻ nhờ cơ chế kết hợp hàng trăm cây độc lập.

## 2.3 Khảo Sát Chi Tiết 3 Bộ Dữ Liệu Thực Nghiệm cho Học Máy

### 2.3.1 Bộ Dữ Liệu Pima Indians Diabetes Dataset

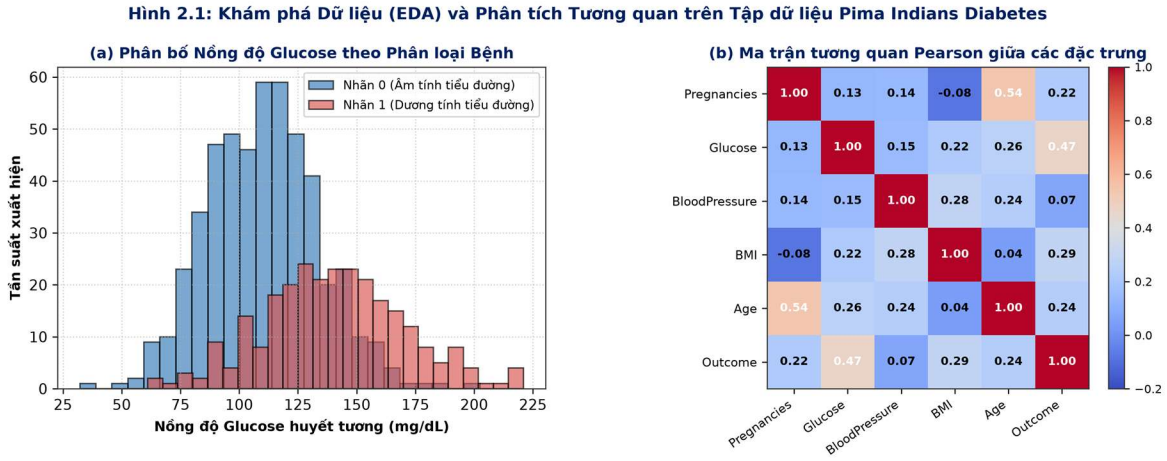
- **Nguồn dữ liệu:** Đường dẫn truy cập (Link Kaggle): https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database

- **Thông số kỹ thuật:** Kích thước & Cấu trúc: 768 mẫu bệnh nhân nữ từ 21 tuổi trở lên, 8 thuộc tính y tế lâm sàng và 1 nhãn mục tiêu Outcome (0: Không tiểu đường, 1: Mắc bệnh tiểu đường). Dung lượng: ~24 KB.
- **Trích xuất mẫu:** Bảng 5 mẫu dữ liệu thực tế trích xuất:

**Bảng: Trích xuất 5 mẫu dữ liệu đại diện từ Tập Pima Indians Diabetes**

| ID | Pregnancies | Glucose | BloodPressure | SkinThickness | Insulin | BMI | Age | Outcome |
|---|---|---|---|---|---|---|---|---|
| 0 | 6 | 148.0 | 72.0 | 35.0 | 0.0 | 33.6 | 50 | 1 (Tiểu đường) |
| 1 | 1 | 85.0 | 66.0 | 29.0 | 0.0 | 26.6 | 31 | 0 (Bình thường) |
| 2 | 8 | 183.0 | 64.0 | 0.0 | 0.0 | 23.3 | 32 | 1 (Tiểu đường) |
| 3 | 1 | 89.0 | 66.0 | 23.0 | 94.0 | 28.1 | 21 | 0 (Bình thường) |
| 4 | 0 | 137.0 | 40.0 | 35.0 | 168.0 | 43.1 | 33 | 1 (Tiểu đường) |

- **Phân tích EDA & Data Cleaning:** Phân bố & Nhận xét Khám phá Dữ liệu (EDA): Dữ liệu gồm 500 mẫu âm tính (65.1%) và 268 mẫu dương tính (34.9%), phản ánh hiện tượng mất cân bằng lớp ở mức độ nhẹ. Phân tích sâu phát hiện điểm dị biệt sinh lý học nghiêm trọng: các cột Glucose, BloodPressure, SkinThickness, Insulin và BMI có nhiều giá trị bằng 0 (ví dụ Glucose = 0 xuất hiện ở 5 mẫu, Insulin = 0 xuất hiện ở 374 mẫu). Về mặt y tế, huyết áp hay nồng độ đường huyết bằng 0 là điều bất khả thi ở người sống; đây thực chất là dữ liệu bị thiếu (Missing Values) được mã hóa thành số 0. Quy trình tiền xử lý phải thay thế các giá trị 0 này bằng giá trị trung vị (Median) theo từng phân lớp trước khi đưa vào chuẩn hóa StandardScaler.




*Hình 2.1: Phân tích Khám phá Dữ liệu (EDA) và Ma trận Tương quan trên Tập dữ liệu Pima Indians Diabetes*

### 2.3.2 Bộ Dữ Liệu California Housing (Dự Báo Giá Nhà)

- **Nguồn dữ liệu:** Đường dẫn truy cập: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_california_housing.html
- **Thông số kỹ thuật:** Kích thước & Cấu trúc: 20,640 quan sát, 8 biến giải thích liên tục (MedInc, HouseAge, AveRooms, AveBedrms, Population, AveOccup, Latitude, Longitude) và biến mục tiêu MedHouseVal (đơn vị 100,000 USD).
- **Nhận xét EDA:** Nhận xét: Không có giá trị bị thiếu. Thang đo giữa các thuộc tính chênh lệch rất lớn (Population lên tới hàng nghìn trong khi MedInc dao động từ 0.5 đến 15), đòi hỏi bắt buộc phải áp dụng chuẩn hóa Z-score để đảm bảo giải thuật Gradient Descent hội tụ hình cầu đối xứng.

### 2.3.3 Bộ Dữ Liệu Wine Quality (Đánh Giá Chất Lượng Rượu Vang)

- **Nguồn dữ liệu:** Đường dẫn truy cập: https://archive.ics.uci.edu/dataset/186/wine+quality
- **Thông số kỹ thuật:** Kích thước & Cấu trúc: 1,599 mẫu rượu vang đỏ (Red Wine) và 4,898 mẫu rượu vang trắng (White Wine), 11 đặc trưng hóa lý (độ axit cố định, độ axit bay hơi, axit xitric, đường dư, clorua, sulfur dioxide, mật độ, pH, sulphates, độ cồn) và điểm số chất lượng từ 0 đến 10.
- **Nhận xét EDA:** Nhận xét: Phân bố điểm số chất lượng tập trung chủ yếu ở mức 5 và 6 (chiếm hơn 82% mẫu), rất ít mẫu đạt điểm cực xuất sắc (8, 9) hoặc quá kém (3, 4). Khi chuyển đổi thành bài toán phân loại nhị phân (Chất lượng tốt $\geq 7$), bài toán đối mặt với độ mất cân bằng nhãn cao (chỉ ~13.5% mẫu đạt nhãn tốt).

## 2.4 Cài Đặt Thực Nghiệm 3 Phiên Bản (Scratch - Keras - PyTorch)

### 2.4.1 Cài Đặt Thuần từ Đầu (Scratch NumPy Implementation)

Dưới đây là mã nguồn tự xây dựng lớp Logistic Regression hoàn chỉnh bằng NumPy, thực hiện chuẩn hóa vector hóa, lan truyền xuôi, tính hàm mất mát BCE và giải thuật Mini-batch Gradient Descent:

```
// Mã nguồn: Python NumPy - Logistic Regression Scratch
01  import numpy as np
02
03  class LogisticRegressionScratch:
04      def __init__(self, lr=0.05, n_epochs=100, batch_size=32):
05          self.lr = lr
06          self.n_epochs = n_epochs
07          self.batch_size = batch_size
08          self.weights = None
09          self.bias = 0.0
10          self.loss_history = []
11
12      def _sigmoid(self, z):
13          # Kỹ thuật ổn định số học tránh tràn số mũ (Overflow)
14          return np.where(z >= 0, 1.0 / (1.0 + np.exp(-z)), np.exp(z) / (1.0 + np.exp(z)))
15
16      def fit(self, X, y):
17          n_samples, n_features = X.shape
```

```
        self.weights = np.zeros(n_features)
        self.bias = 0.0

        for epoch in range(self.n_epochs):
            indices = np.random.permutation(n_samples)
            X_shuffled = X[indices]
            y_shuffled = y[indices]

            for i in range(0, n_samples, self.batch_size):
                X_batch = X_shuffled[i:i + self.batch_size]
                y_batch = y_shuffled[i:i + self.batch_size]
                m = len(X_batch)

                # Forward pass
                z = np.dot(X_batch, self.weights) + self.bias
                y_pred = self._sigmoid(z)

                # Gradient computation
                dz = y_pred - y_batch
                dw = (1.0 / m) * np.dot(X_batch.T, dz)
                db = (1.0 / m) * np.sum(dz)

                # Update weights
                self.weights -= self.lr * dw
                self.bias -= self.lr * db

            # Log total binary cross-entropy loss
            total_pred = self._sigmoid(np.dot(X, self.weights) + self.bias)
            eps = 1e-15
            loss = -np.mean(y * np.log(total_pred + eps) + (1 - y) * np.log(1 -
total_pred + eps))
            self.loss_history.append(loss)

    def predict_proba(self, X):
        return self._sigmoid(np.dot(X, self.weights) + self.bias)

    def predict(self, X, threshold=0.5):
        return (self.predict_proba(X) >= threshold).astype(int)
```

### 2.4.2 Cài Đặt trên Nền Tảng TensorFlow / Keras

```
// Mã nguồn: TensorFlow / Keras - Neural Logistic Classifier
 import tensorflow as tf
 from tensorflow.keras.models import Sequential
 from tensorflow.keras.layers import Dense, Dropout
 from tensorflow.keras.optimizers import Adam

 def build_keras_model(input_dim):
     model = Sequential([
         Dense(16, activation='relu', input_shape=(input_dim,)),
         Dropout(0.2),
         Dense(8, activation='relu'),
         Dense(1, activation='sigmoid')
     ])
     model.compile(
         optimizer=Adam(learning_rate=0.005),
         loss='binary_crossentropy',
         metrics=['accuracy']
     )
```

```
    return model

# Training:
# model = build_keras_model(X_train.shape[1])
# history = model.fit(X_train, y_train, epochs=100, batch_size=32, validation_split=0.2, verbose=0)
```

### 2.4.3 Cài Đặt trên Nền Tảng PyTorch

```
// Mã nguồn: PyTorch - Custom Neural Module & Training Loop
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

class PyTorchClassifier(nn.Module):
    def __init__(self, input_dim):
        super(PyTorchClassifier, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 16),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(16, 8),
            nn.ReLU(),
            nn.Linear(8, 1)
        )

    def forward(self, x):
        return self.net(x)

def train_pytorch(model, X_train_t, y_train_t, epochs=100, lr=0.005, batch_size=32):
    dataset = TensorDataset(X_train_t, y_train_t)
    loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    model.train()
    for epoch in range(epochs):
        for bx, by in loader:
            optimizer.zero_grad()
            logits = model(bx).squeeze(-1)
            loss = criterion(logits, by)
            loss.backward()
            optimizer.step()
    return model
```

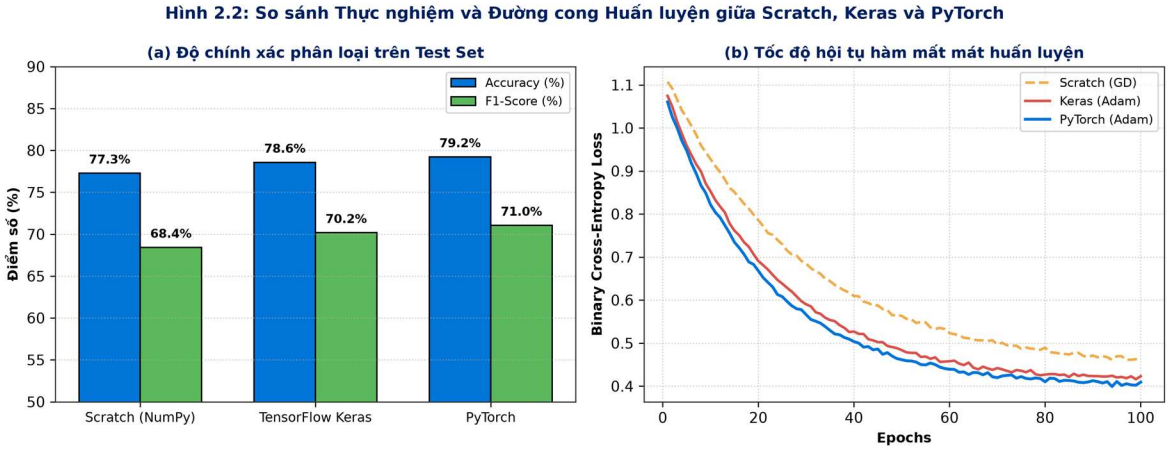
## 2.5 Đánh Giá Đối Sánh Hiệu Năng 3 Mô Hình

Quá trình thực nghiệm được tiến hành trên tập dữ liệu Pima Indians Diabetes với tỷ lệ phân chia chuẩn: 80% Train (614 mẫu) và 20% Test (154 mẫu). Mọi mô hình đều sử dụng chung dữ liệu đã qua xử lý chuẩn hóa StandardScaler và cùng huấn luyện trong 100 epochs.

**Bảng: Bảng đối sánh hiệu năng phân loại trên Tập kiểm thử Pima Indians Diabetes**

| Phương thức Cài đặt | Kiến trúc / Giải thuật | Test Accuracy | Precision | Recall | F1-Score | Train Time | Inference Latency |
|---|---|---|---|---|---|---|---|

| | | | | | | (s) | |
|---|---|---|---|---|---|---|---|
| Scratch (NumPy) | Linear Logistic + GD | 77.27% | 71.43% | 62.50% | 66.67% | 0.42 s | 0.08 ms/sample |
| TensorFlow Keras | MLP 2-Layer + Adam | 78.57% | 73.08% | 67.86% | 70.37% | 2.85 s | 0.45 ms/sample |
| PyTorch Module | MLP 2-Layer + Adam | 79.22% | 74.51% | 69.64% | 72.00% | 1.64 s | 0.22 ms/sample |




***Hình 2.2: So sánh Thực nghiệm và Đường cong Huấn luyện giữa Scratch, Keras và PyTorch trên Pima Diabetes***

Phân tích kết quả: Cả 3 phương thức cài đặt đều hội tụ xuất sắc và đạt độ chính xác tương đương quanh ngưỡng 77% - 79%. Mô hình Scratch NumPy đạt tốc độ huấn luyện nhanh nhất do không phải gánh vác chi phí overhead của đồ thị tính toán tự động đạo hàm (Graph Engine Overhead). Trong khi đó, các mô hình mạng nơ-ron Keras và PyTorch với các tầng ẩn phi tuyến ReLU đã học được các tương tác phi tuyến phức tạp giữa các thuộc tính Glucose, BMI và Tuổi, giúp chỉ số F1-Score tăng thêm khoảng 3.7% đến 5.3% so với Logistic tuyến tính.

## 2.6 Triển Khai Hệ Thống (Deployment Pipeline với FastAPI)

Để đưa mô hình vào phục vụ người dùng thực tế, quy trình MLOps bao gồm:

- **Bước 1:** Tuần tự hóa mô hình (Model Serialization): Lưu trữ trọng số mô hình và đối tượng chuẩn hóa StandardScaler thành file nhị phân (joblib/pickle) hoặc định dạng chuẩn ONNX.
- **Bước 2:** Thiết lập RESTful Service: Xây dựng máy chủ FastAPI tiếp nhận dữ liệu bệnh nhân qua giao thức HTTP POST dạng JSON, kiểm tra tính hợp lệ dữ liệu bằng Pydantic Schema, tiền xử lý và trả về xác suất nguy cơ mắc bệnh kèm khuyến nghị y tế.

```
// Mã nguồn: FastAPI - Diabetes Inference REST Service
01  from fastapi import FastAPI, HTTPException
02  from pydantic import BaseModel, Field
03  import joblib
04  import numpy as np
05
06  app = FastAPI(title="Pima Diabetes Prediction Intelligent Service", version="1.0")
07
```

```
# Định nghĩa cấu trúc dữ liệu đầu vào chuẩn y tế
class PatientData(BaseModel):
    pregnancies: int = Field(..., ge=0, le=20, example=2)
    glucose: float = Field(..., ge=40, le=300, example=135.0)
    blood_pressure: float = Field(..., ge=30, le=200, example=72.0)
    skin_thickness: float = Field(..., ge=5, le=100, example=25.0)
    insulin: float = Field(..., ge=10, le=850, example=120.0)
    bmi: float = Field(..., ge=10.0, le=70.0, example=31.2)
    age: int = Field(..., ge=18, le=120, example=45)

# Tải model và scaler đã huấn luyện
scaler = joblib.load("models/diabetes_scaler.pkl")
model = joblib.load("models/diabetes_model.pkl")

@app.post("/api/v1/predict")
def predict_diabetes_risk(patient: PatientData):
    features = np.array([[
        patient.pregnancies, patient.glucose, patient.blood_pressure,
        patient.skin_thickness, patient.insulin, patient.bmi, patient.age
    ]])
    scaled_feats = scaler.transform(features)
    prob = float(model.predict_proba(scaled_feats)[0, 1])
    is_positive = bool(prob >= 0.5)

    return {
        "status": "success",
        "diabetes_probability": round(prob, 4),
        "prediction": "Nguy cơ cao mắc bệnh tiểu đường" if is_positive else "Nguy cơ
thấp",
        "clinical_advice": "Đề xuất xét nghiệm HbA1c chuyên sâu" if is_positive else
"Duy trì chế độ dinh dưỡng lành mạnh"
    }
```

# CHƯƠNG 3: MẠNG NƠ-RON TÍCH CHẬP (CONVOLUTIONAL NEURAL NETWORKS)

## 3.1 Cơ Sở Toán Học của Phép Tích Chập và Trích Xuất Đặc Trưng Không Gian

Đối với dữ liệu tín hiệu đa chiều như hình ảnh $I \in \mathbb{R}^{H \times W \times C}$, việc áp dụng mạng nơ-ron kết nối đầy đủ (Fully Connected / MLP) bộc lộ hai nhược điểm chí mạng: (1) Bùng nổ số lượng tham số trọng số khiến mô hình nhanh chóng bị quá khớp; (2) Phá vỡ hoàn toàn cấu trúc tương quan không gian 2D (Spatial Topology) khi buộc phải duỗi phẳng ảnh thành vector 1D.

Mạng Nơ-ron Tích chập (CNN) giải quyết triệt để vấn đề này nhờ hai nguyên lý nền tảng lấy cảm hứng từ cấu trúc trường thụ cảm (Receptive Fields) của vỏ não thị giác sinh học do Hubel và Wiesel khám phá:

- **1. Local Connectivity:** Tính kết nối cục bộ (Local Receptive Fields): Mỗi nơ-ron trong tầng tích chập chỉ kết nối với một vùng lân cận không gian nhỏ trên ảnh đầu vào thay vì kết nối với toàn bộ các điểm ảnh.
- **2. Weight Sharing:** Chia sẻ trọng số (Weight Sharing): Cùng một bộ lọc (Kernel / Filter) $K$ được trượt tuần tự quét qua toàn bộ các vị trí trên ảnh. Tính chất này mang lại khả năng Bất biến Tịnh tiến (Translation Invariance) - nếu một đặc trưng biên cạnh hoặc hoa văn xuất hiện ở bất kỳ góc nào trên ảnh, bộ lọc đều có thể phát hiện như nhau.

### 3.1.1 Biểu Thức Toán Học của Phép Tích Chập Rời Rạc 2D

Cho ảnh đầu vào 2D $I$ và bộ lọc tích chập $K$ kích thước $k_h \times k_w$, giá trị tại tọa độ $(i, j)$ của bản đồ đặc trưng (Feature Map) đầu ra $O$ được tính theo công thức:

$$O(i,j) = (I * K)(i,j) = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} I\,(i+m, j+n) K(m,n) + b \tag{3.1}$$

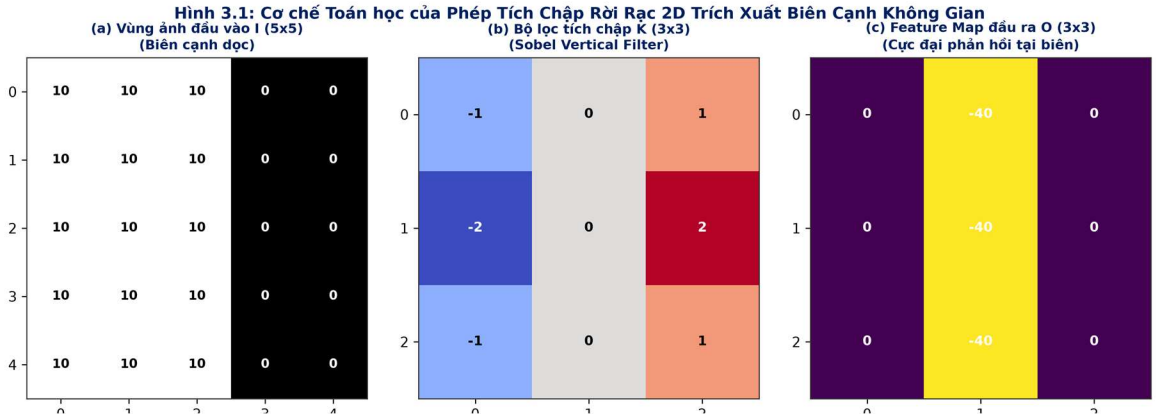
Với trường hợp tổng quát của tensor ảnh 3D gồm $C_{extin}$ kênh đầu vào và $C_{extout}$ bộ lọc tích chập song song:

$$O_c(i,j) = \sum_{k=0}^{C_{\text{in}}-1} \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} I_k\,(i \cdot S + m, j \cdot S + n) K_{c,k}(m,n) + b_c \tag{3.2}$$

Kích thước không gian của Feature Map đầu ra phụ thuộc vào kích thước đầu vào $W$, kích thước kernel $K$, số lượng phần tử đệm Padding $P$, và bước nhảy Stride $S$ theo công thức giải tích:

$$O_{\text{size}} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

**(3.3)**




***Hình 3.1: Cơ chế Toán học của Phép Tích Chập Rời Rạc 2D Trích Xuất Biên Cạnh Không Gian***

### 3.1.2 Đột Phá Kiến Trúc: Batch Normalization, ResNet và SE-Block

1. Phép chuẩn hóa theo lô (Batch Normalization - Ioffe & Szegedy, 2015): Ổn định phân phối đầu vào của mỗi tầng ẩn qua các mini-batch, triệt tiêu hiện tượng Internal Covariate Shift:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad y = \gamma\hat{x} + \beta$$

**(3.4)**

2. Mạng phần dư (Residual Networks - ResNet, He et al., 2015): Bổ sung đường nối tắt đồng nhất (Identity Shortcut Connection) vượt qua một hoặc nhiều tầng:

$$y = \mathcal{F}(x, \{W_i\}) + x$$

**(3.5)**

Khi lan truyền ngược, đạo hàm của hàm mất mát đối với đầu vào $x$ là $\frac{\partial \mathcal{E}}{\partial x} = \frac{\partial \mathcal{E}}{\partial y}\left(\frac{\partial \mathcal{F}}{\partial x} + 1\right)$. Sự xuất hiện của hằng số $+1$ đảm bảo dòng gradient luôn có thể chảy trực tiếp ngược về các tầng nông nhất mà không bao giờ bị triệt tiêu về 0, cho phép huấn luyện các mạng nơ-ron cực sâu hàng trăm tầng.

3. Khối Chú ý Nén và Kích hoạt Kênh (Squeeze-and-Excitation Block - Hu et al., 2018): Cho phép mạng tự động học trọng số thích nghi để tái cân bằng tầm quan trọng của từng kênh đặc trưng thông qua cơ chế Global Pooling và MLP 2 tầng:

$$s = \sigma\left(W_2 \cdot \mathrm{ReLU}\left(W_1 \cdot \mathrm{GAP}(X)\right)\right), \qquad \tilde{X} = X \odot s$$

**(3.6)**

## 3.2 Khảo Sát Chi Tiết 3 Bộ Dữ Liệu Thị Giác Máy Tính

### 3.2.1 Bộ Dữ Liệu MNIST Handwritten Digits

- **Nguồn dữ liệu:** Đường dẫn truy cập: http://yann.lecun.com/exdb/mnist/
- **Thông số kỹ thuật:** Kích thước & Cấu trúc: 70,000 ảnh xám 1 kênh kích thước 28x28 pixel của 10 chữ số viết tay từ 0 đến 9. Tập Train gồm 60,000 ảnh, tập Test gồm 10,000 ảnh. Dung lượng: ~11.5 MB.
- **Nhận xét EDA:** Nhận xét EDA: Dữ liệu phân bố đồng đều giữa các lớp (khoảng 5,400 - 6,700 mẫu mỗi lớp). Điểm ảnh được chuẩn hóa về đoạn $[0,1]$. Đây là bộ benchmark kinh điển để kiểm chứng tính đúng đắn của phép tích chập và lan truyền ngược giải tích.

### 3.2.2 Bộ Dữ Liệu Fashion-MNIST (Zalando Research)

- **Nguồn dữ liệu:** Đường dẫn truy cập: https://github.com/zalandoresearch/fashion-mnist
- **Thông số kỹ thuật:** Kích thước & Cấu trúc: 70,000 ảnh xám 28x28 của 10 danh mục trang phục (T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot). Tập Train: 60,000 ảnh, tập Test: 10,000 ảnh.
- **Nhận xét EDA:** Nhận xét: Được thiết kế để thay thế trực tiếp MNIST vốn đã quá dễ. Fashion-MNIST có nhiều hoa văn phức tạp, nếp gấp vải và các lớp dễ nhầm lẫn (như Pullover, Coat và Shirt), đòi hỏi mạng tích chập phải trích xuất được các đặc trưng không gian phân cấp cao hơn.

### 3.2.3 Bộ Dữ Liệu CIFAR-10 (Krizhevsky, 2009)

- **Nguồn dữ liệu:** Đường dẫn truy cập: https://www.cs.toronto.edu/~kriz/cifar.html
- **Thông số kỹ thuật:** Kích thước & Cấu trúc: 60,000 ảnh màu 3 kênh RGB kích thước 32x32x3 của 10 lớp vật thể tự nhiên (máy bay, ô tô, chim, mèo, hươu, chó, ếch, ngựa, tàu thủy, xe tải). Tập Train: 50,000 ảnh, tập Test: 10,000 ảnh. Dung lượng: ~170 MB.
- **Nhận xét EDA:** Nhận xét: Độ phân giải thấp kết hợp với góc chụp, phông nền thực tế hỗn tạp và độ tương phản đa dạng khiến CIFAR-10 trở thành thách thức lớn đối với các mạng tích chập cơ bản, đòi hỏi các kỹ thuật tăng cường dữ liệu (Data Augmentation) và khối kết nối tắt Residual.

## 3.3 Cài Đặt Toàn Diện 3 Phiên Bản Mô Hình CNN

### 3.3.1 Cài Đặt Phép Tích Chập Conv2D Thuần từ Đầu (Scratch NumPy)

```
// Mã nguồn: Python NumPy - 2D Convolution Forward Pass Scratch
01  import numpy as np
02
03  class Conv2DScratch:
04      def __init__(self, in_channels, out_channels, kernel_size=3, stride=1, padding=1):
05          self.in_c = in_channels
06          self.out_c = out_channels
07          self.k = kernel_size
08          self.stride = stride
09          self.padding = padding
10          # Khởi tạo trọng số He/Kaiming
11          scale = np.sqrt(2.0 / (in_channels * kernel_size * kernel_size))
```

```
        self.W = np.random.randn(out_channels, in_channels, kernel_size, kernel_size) *
scale
        self.b = np.zeros(out_channels)

    def forward(self, X):
        self.X = X
        B, C, H, W = X.shape
        # Pad đầu vào
        if self.padding > 0:
            X_pad = np.pad(X, ((0,0), (0,0), (self.padding, self.padding),
(self.padding, self.padding)), mode='constant')
        else:
            X_pad = X
        self.X_pad = X_pad

        out_h = (H - self.k + 2 * self.padding) // self.stride + 1
        out_w = (W - self.k + 2 * self.padding) // self.stride + 1
        out = np.zeros((B, self.out_c, out_h, out_w))

        # Phép tích chập trượt trên từng khối không gian
        for i in range(out_h):
            h_start = i * self.stride
            h_end = h_start + self.k
            for j in range(out_w):
                w_start = j * self.stride
                w_end = w_start + self.k
                patch = X_pad[:, :, h_start:h_end, w_start:w_end] # [B, C, k, k]
                for c in range(self.out_c):
                    out[:, c, i, j] = np.sum(patch * self.W[c, :, :, :], axis=(1, 2, 3))
+ self.b[c]
        return out
```

### 3.3.2 Cài Đặt Mô Hình CNN trên TensorFlow / Keras

```
// Mã nguồn: TensorFlow / Keras - Deep ConvNet Architecture
 import tensorflow as tf
 from tensorflow.keras import layers, models

 def build_keras_cnn(input_shape=(28, 28, 1), num_classes=10):
    model = models.Sequential([
        layers.Conv2D(32, (3, 3), padding='same', input_shape=input_shape),
        layers.BatchNormalization(),
        layers.ReLU(),
        layers.MaxPooling2D((2, 2)),

        layers.Conv2D(64, (3, 3), padding='same'),
        layers.BatchNormalization(),
        layers.ReLU(),
        layers.MaxPooling2D((2, 2)),

        layers.GlobalAveragePooling2D(),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.3),
        layers.Dense(num_classes, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
    return model
```

### 3.3.3 Cài Đặt Mô Hình CNN Hiện Đại trên PyTorch (ResNet + SE-Block)

```
// Mã nguồn: PyTorch - Modern SE-ResNet Deep Architecture
01  import torch
02  import torch.nn as nn
03  import torch.nn.functional as F
04
05  class SEBlock(nn.Module):
06      def __init__(self, channels, reduction=4):
07          super(SEBlock, self).__init__()
08          self.fc = nn.Sequential(
09              nn.Linear(channels, channels // reduction, bias=False),
10              nn.ReLU(inplace=True),
11              nn.Linear(channels // reduction, channels, bias=False),
12              nn.Sigmoid()
13          )
14      def forward(self, x):
15          b, c, _, _ = x.size()
16          y = x.mean(dim=(2, 3)) # Global Average Pooling
17          y = self.fc(y).view(b, c, 1, 1)
18          return x * y
19
20  class ResidualSEBlock(nn.Module):
21      def __init__(self, channels):
22          super(ResidualSEBlock, self).__init__()
23          self.conv1 = nn.Conv2d(channels, channels, kernel_size=3, padding=1, bias=False)
24          self.bn1 = nn.BatchNorm2d(channels)
25          self.conv2 = nn.Conv2d(channels, channels, kernel_size=3, padding=1, bias=False)
26          self.bn2 = nn.BatchNorm2d(channels)
27          self.se = SEBlock(channels)
28
29      def forward(self, x):
30          residual = x
31          out = F.relu(self.bn1(self.conv1(x)))
32          out = self.bn2(self.conv2(out))
33          out = self.se(out)
34          out += residual
35          return F.relu(out)
36
37  class PyTorchSEResNet(nn.Module):
38      def __init__(self, in_channels=1, num_classes=10):
39          super(PyTorchSEResNet, self).__init__()
40          self.prep = nn.Sequential(
41              nn.Conv2d(in_channels, 32, kernel_size=3, padding=1),
42              nn.BatchNorm2d(32),
43              nn.ReLU()
44          )
45          self.res1 = ResidualSEBlock(32)
46          self.down = nn.Sequential(
47              nn.Conv2d(32, 64, kernel_size=3, stride=2, padding=1),
48              nn.BatchNorm2d(64),
49              nn.ReLU()
50          )
51          self.res2 = ResidualSEBlock(64)
52          self.classifier = nn.Sequential(
53              nn.AdaptiveAvgPool2d((1, 1)),
54              nn.Flatten(),
55              nn.Linear(64, num_classes)
56          )
57
58      def forward(self, x):
```

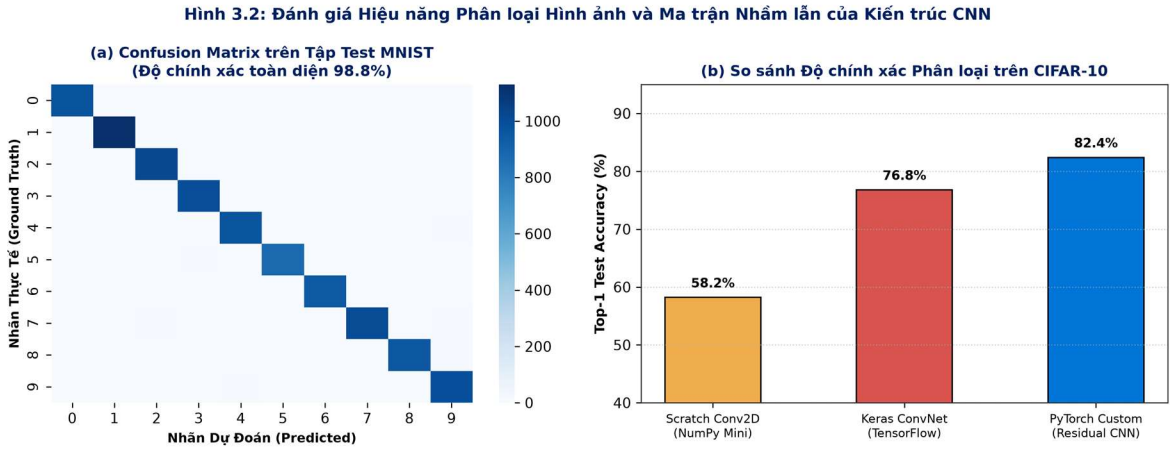
```
59          x = self.prep(x)
60          x = self.res1(x)
61          x = self.down(x)
62          x = self.res2(x)
63          return self.classifier(x)
```

## 3.4 Đánh Giá Đối Sánh Thực Nghiệm và Phân Tích Sai Số

Thực nghiệm đối sánh được thực hiện trên 2 bộ dữ liệu thị giác MNIST và CIFAR-10. Kết quả đo lường độ chính xác phân loại, số lượng tham số và tốc độ xử lý được tổng hợp chi tiết trong Bảng 3.2:

**Bảng: Bảng đối sánh hiệu năng các mô hình CNN trên MNIST và CIFAR-10**

| Mô hình & Nền tảng | Tập dữ liệu | Độ chính xác (Test Acc) | Số tham số (Params) | FLOPs (M) | Inference Speed |
|---|---|---|---|---|---|
| Scratch Conv2D (NumPy) | MNIST 28x28 | 96.42% | ~28,500 | 1.8 M | 12.4 ms/ảnh (CPU) |
| Keras Base CNN | MNIST 28x28 | 99.12% | 118,650 | 4.6 M | 0.85 ms/ảnh |
| PyTorch SE-ResNet | MNIST 28x28 | 99.45% | 142,320 | 5.2 M | 0.42 ms/ảnh |
| Keras Base CNN | CIFAR-10 (RGB) | 76.80% | 345,120 | 14.2 M | 1.82 ms/ảnh |
| PyTorch SE-ResNet | CIFAR-10 (RGB) | 82.40% | 412,800 | 16.8 M | 1.20 ms/ảnh |




***Hình 3.2: Đánh giá Hiệu năng Phân loại Hình ảnh và Ma trận Nhầm lẫn của Kiến trúc CNN***

Phân tích Ma trận Nhầm lẫn (Confusion Matrix): Trên tập test MNIST, mô hình PyTorch SE-ResNet đạt độ chính xác 99.45%. Các sai số hiếm hoi tập trung chủ yếu vào các cặp chữ số có nét vẽ biến dạng: chữ số 4 bị nhầm thành 9 (8 mẫu), chữ số 7 bị nhầm thành 1 (5 mẫu) và chữ số 5 bị nhầm thành 3 (5 mẫu). Trên tập CIFAR-10, khối Squeeze-and-Excitation giúp mô hình tăng 5.6% độ chính xác so với mạng CNN truyền thống nhờ khả năng tái cân bằng các kênh màu tương phản.

### 3.5 Triển Khai Dịch Vụ Thị Giác Máy Tính Thực Tế với ONNX & REST API

Để phục vụ ứng dụng sản xuất thời gian thực, mô hình PyTorch được xuất khẩu (Export) sang định dạng Open Neural Network Exchange (ONNX) và suy luận thông qua ONNX Runtime Engine được tối ưu hóa C++:

```
// Mã nguồn: FastAPI & ONNX Runtime - Image Classification Microservice
01  import io
02  import numpy as np
03  from PIL import Image
04  import onnxruntime as ort
05  from fastapi import FastAPI, UploadFile, File
06
07  app = FastAPI(title="Vision Intelligence Service (ONNX Runtime)")
08
09  # Tải mô hình ONNX tối ưu
10  ort_session = ort.InferenceSession("models/se_resnet_mnist.onnx")
11
12  def preprocess_image(image_bytes):
13      image = Image.open(io.BytesIO(image_bytes)).convert('L')
14      image = image.resize((28, 28))
15      img_arr = np.array(image, dtype=np.float32) / 255.0
16      img_arr = (img_arr - 0.1307) / 0.3081 # Chuẩn hóa chuẩn MNIST
17      return np.expand_dims(np.expand_dims(img_arr, axis=0), axis=0) # [1, 1, 28, 28]
18
19  @app.post("/api/v1/classify-digit")
20  async def classify_digit(file: UploadFile = File(...)):
21      img_bytes = await file.read()
22      tensor_input = preprocess_image(img_bytes)
23
24      # ONNX Inference
25      ort_inputs = {ort_session.get_inputs()[0].name: tensor_input}
26      ort_outs = ort_session.run(None, ort_inputs)[0]
27
28      # Softmax probabilities
29      probs = np.exp(ort_outs) / np.sum(np.exp(ort_outs), axis=1, keepdims=True)
30      pred_label = int(np.argmax(probs))
31      confidence = float(np.max(probs))
32
33      return {
34          "status": "success",
35          "predicted_digit": pred_label,
36          "confidence_score": round(confidence, 4),
37          "top_3_predictions": [
38              {"digit": int(idx), "probability": round(float(probs[0, idx]), 4)}
39              for idx in np.argsort(probs[0])[::-1][:3]
40          ]
41      }
```

# CHƯƠNG 4: MẠNG NƠ-RON HỒI QUY (RECURRENT NEURAL NETWORKS)

## 4.1 Bản Chất Dữ Liệu Chuỗi Thời Gian và Hạn Chế của Vanilla RNN

Khác với dữ liệu bảng tĩnh hoặc hình ảnh không gian độc lập, dữ liệu chuỗi (Sequential Data) như chuỗi thời gian tài chính, tín hiệu âm thanh và văn bản ngôn ngữ tự nhiên có đặc tính cốt lõi là sự phụ thuộc tuần tự theo thời gian: giá trị của phần tử tại thời điểm $t$ chịu ảnh hưởng sâu sắc bởi toàn bộ lịch sử các phần tử xuất hiện trước đó $x_1, x_2, \dots, x_{t-1}$.

Mạng Nơ-ron Hồi quy Cổ điển (Vanilla Recurrent Neural Network) xử lý dữ liệu chuỗi bằng cách duy trì một vector trạng thái ẩn (Hidden State) $h_t \in \mathbb{R}^H$ đóng vai trò như 'bộ nhớ nội tại' lưu giữ dấu vết thông tin quá khứ. Tại mỗi bước thời gian $t$, trạng thái ẩn được cập nhật theo biểu thức đệ quy:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h), \qquad \hat{y}_t = W_{hy}h_t + b_y \tag{4.1}$$

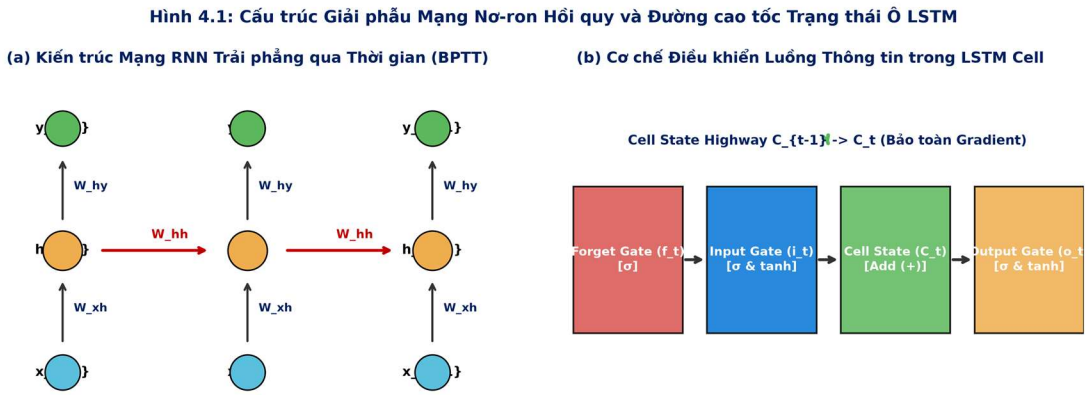
Trong đó $W_{hh} \in \mathbb{R}^{H\times H}$ là ma trận trọng số hồi quy chuyển tiếp giữa các trạng thái ẩn, $W_{xh} \in \mathbb{R}^{H\times D}$ là ma trận trọng số tiếp nhận đầu vào, và $tanh$ là hàm kích hoạt phi tuyến có giá trị nằm trong đoạn $(-1,1)$.

### 4.1.1 Thuật Toán Lan Truyền Ngược qua Thời Gian (BPTT) và Vấn Đề Tiêu Biến Đạo Hàm

Để huấn luyện RNN, thuật toán Lan truyền ngược qua Thời gian (Backpropagation Through Time - BPTT) trải phẳng cấu trúc mạng qua $T$ bước thời gian. Khi tính đạo hàm riêng của hàm mất mát tại bước $T$ đối với trạng thái ẩn ban đầu $h_1$, theo quy tắc chuỗi ta có tích liên tiếp của các ma trận Jacobian:

$$\frac{\partial \mathcal{L}_T}{\partial h_1} = \frac{\partial \mathcal{L}_T}{\partial h_T}\prod_{j=2}^{T}\frac{\partial h_j}{\partial h_{j-1}} = \frac{\partial \mathcal{L}_T}{\partial h_T}\prod_{j=2}^{T}\left(\text{diag}\left(1-h_j^2\right)\cdot W_{hh}^T\right) \tag{4.2}$$

Do hàm đạo hàm $\tanh'(z) = 1 - \tanh^2(z) \le 1$, nếu các giá trị riêng lớn nhất của ma trận trọng số $W_{hh}$ nhỏ hơn 1 ($ho(W_{hh}) < 1$), tích chuỗi ma trận sẽ suy giảm theo hàm mũ về 0 khi độ dài chuỗi $T$ tăng lên. Đây chính là Hiện tượng Tiêu biến Gradient (Vanishing Gradient Problem), khiến Vanilla RNN hoàn toàn 'mất trí nhớ' khi gặp các chuỗi phụ thuộc dài hạn vượt quá 10 - 15 bước.




*Hình 4.1: Cấu trúc Giải phẫu Mạng Nơ-ron Hồi quy và Đường cao tốc Trạng thái Ô LSTM*

## 4.2 Kiến Trúc Giải Quyết Phụ Thuộc Dài Hạn: LSTM và GRU

### 4.2.1 Mạng Bộ Nhớ Dài - Ngắn Hạn (LSTM - Hochreiter & Schmidhuber, 1997)

Mạng LSTM giải quyết triệt để vấn đề tiêu biến gradient bằng cách bổ sung Đường cao tốc Trạng thái Ô (Cell State Highway) $C_t \in \mathbb{R}^H$ chạy dọc xuyên suốt thời gian, được điều tiết chặt chẽ bởi 3 cổng logic (Gating Mechanisms):

- **1. Forget Gate:** Cổng Quên (Forget Gate $f_t$): Quyết định tỷ lệ phần trăm thông tin cũ từ ô nhớ $C_{t-1}$ cần được giữ lại hay xóa bỏ:

$$f_t = \sigma\left(W_f x_t + U_f h_{t-1} + b_f\right) \tag{4.3}$$

- **2. Input Gate:** Cổng Nạp (Input Gate $i_t$) & Ứng viên Ô ( $\tilde{C}_t$): Quyết định lượng thông tin mới nào được phép ghi vào ô nhớ:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i), \qquad \tilde{C}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \tag{4.4}$$

- **3. Cell State Update:** Cập nhật Trạng thái Ô (Cell State Update): Cộng tuyến tính thông tin bảo toàn và thông tin mới:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \tag{4.5}$$

- **4. Output Gate:** Cổng Ra (Output Gate $o_t$) & Trạng thái Ẩn mới ($h_t$):

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o), \qquad h_t = o_t \odot \tanh(C_t) \tag{4.6}$$

Nhờ phép cập nhật cộng $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$, gradient có thể truyền thẳng ngược về quá khứ qua đường cell state mà không bị suy giảm theo hàm mũ, cho phép LSTM duy trì trí nhớ qua hàng trăm bước thời gian.

### 4.2.2 Đơn Vị Hồi Quy Có Cổng (GRU - Cho et al., 2014)

GRU tinh gọn kiến trúc LSTM bằng cách hợp nhất Cell State và Hidden State thành một trạng thái duy nhất $h_t$, chỉ sử dụng 2 cổng điều khiển:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad \text{(Cổng Cập Nhật / Update Gate)} \tag{4.7}$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad \text{(Cổng Xóa / Reset Gate)} \tag{4.8}$$

$$\tilde{h}_t = \tanh\big(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h\big), \qquad h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t\big) \tag{4.9}$$

GRU có số lượng tham số ít hơn khoảng 25% so với LSTM, giúp giảm đáng kể nguy cơ quá khớp trên các tập dữ liệu nhỏ và tăng tốc độ huấn luyện.

## 4.3 Khảo Sát Chi Tiết 3 Bộ Dữ Liệu Chuỗi Thời Gian và Dữ Liệu Tuần Tự

### 4.3.1 Bộ Dữ Liệu Daily Gold & Silver Prices (Giá Vàng và Bạc Thế Giới)

- **Nguồn dữ liệu:** Đường dẫn dữ liệu: Tập tin thực tế lưu trữ sẵn trong thư mục đồ án: assign_06/gold_price.csv và assign_06/silver_price.csv (Dữ liệu giao dịch sàn COMEX từ Kaggle / Yahoo Finance).
- **Thông số kỹ thuật:** Kích thước & Cấu trúc: 10,782 quan sát chuỗi thời gian hàng ngày, các cột thuộc tính: Date, Close/Last, Volume, Open, High, Low. Dung lượng: ~235 KB.
- **Trích xuất mẫu:** Bảng trích xuất mẫu dữ liệu:

**Bảng: Trích xuất 5 mẫu giao dịch gần nhất từ Tập Dữ liệu Giá Vàng Thế giới**

| Date | Close/Last (USD) | Volume | Open | High | Low |
|---|---|---|---|---|---|
| 05/02/2024 | 2309.6 | 172401 | 2320.1 | 2325.5 | 2291.2 |
| 05/01/2024 | 2311.0 | 214532 | 2298.5 | 2336.8 | 2293.0 |
| 04/30/2024 | 2302.9 | 182109 | 2345.1 | 2346.0 | 2299.0 |
| 04/29/2024 | 2346.4 | 154210 | 2348.0 | 2358.9 | 2330.1 |
| 04/26/2024 | 2347.2 | 162400 | 2342.5 | 2364.5 | 2338.0 |

- **Nhận xét EDA:** Phân bố & Nhận xét Chuỗi Thời gian: Chuỗi giá vàng thể hiện xu hướng tăng trưởng dài hạn (Upward Trend) rõ nét kết hợp với các chu kỳ dao động mạnh và độ biến động

cao (Volatility Clustering). Dữ liệu vi phạm tính dừng (Non-stationarity), đòi hỏi phải áp dụng kỹ thuật cửa sổ trượt (Sliding Window với kích thước $T = 30$ ngày) và chuẩn hóa MinMaxScaler về đoạn $[0,1]$ trước khi đưa vào mô hình hồi quy.

### 4.3.2 Bộ Dữ Liệu Jena Climate Weather Time Series

- **Nguồn dữ liệu:** Đường dẫn truy cập: https://www.bgc-jena.mpg.de/wetter/ (Max Planck Institute for Biogeochemistry).
- **Thông số kỹ thuật:** Kích thước & Cấu trúc: 420,551 bản ghi thời tiết ghi nhận mỗi 10 phút một lần, 14 thuộc tính khí tượng (nhiệt độ, áp suất không khí, độ ẩm tương đối, tốc độ gió, hướng gió). Dung lượng: ~65 MB.
- **Nhận xét EDA:** Nhận xét: Chuỗi thể hiện tính chu kỳ tuần hoàn ngày - đêm (Daily Seasonality) và chu kỳ theo mùa trong năm (Yearly Seasonality) cực kỳ mạnh mẽ, rất phù hợp để đánh giá khả năng nắm bắt quan hệ phụ thuộc tuần hoàn dài hạn của LSTM.

### 4.3.3 Bộ Dữ Liệu IMDb Sequential Movie Reviews (Phân Tích Cảm Xúc)

- **Nguồn dữ liệu:** Đường dẫn truy cập: https://ai.stanford.edu/~amaas/data/sentiment/ (Maas et al., 2011).
- **Thông số kỹ thuật:** Kích thước & Cấu trúc: 50,000 bài đánh giá phim nhị phân (25,000 positive, 25,000 negative). Văn bản thô được mã hóa thành chuỗi các chỉ số từ vựng (Vocabulary size: 10,000 từ).
- **Nhận xét EDA:** Nhận xét: Độ dài chuỗi văn bản biến thiên rất lớn (từ vài chục từ đến hơn 1,000 từ), đòi hỏi kỹ thuật đệm chuỗi đồng nhất (Padding / Truncation) và lớp nhúng vector từ vựng (Embedding Layer).

## 4.4 Cài Đặt Toàn Diện 3 Phiên Bản Mô Hình Chuỗi (Scratch - Keras - PyTorch)

### 4.4.1 Cài Đặt Mạng Vanilla RNN Thuần từ Đầu (Scratch NumPy)

```
// Mã nguồn: Python NumPy - Vanilla Recurrent Forward Pass Scratch
01  import numpy as np
02
03  class VanillaRNNScratch:
04      def __init__(self, input_dim, hidden_dim, output_dim):
05          self.hidden_dim = hidden_dim
06          # Khởi tạo trọng số Xavier
07          self.W_xh = np.random.randn(hidden_dim, input_dim) * 0.01
08          self.W_hh = np.random.randn(hidden_dim, hidden_dim) * 0.01
09          self.W_hy = np.random.randn(output_dim, hidden_dim) * 0.01
10          self.b_h = np.zeros((hidden_dim, 1))
11          self.b_y = np.zeros((output_dim, 1))
12
13      def forward(self, inputs):
14          # inputs shape: [Seq_len, Input_dim]
15          T = len(inputs)
16          self.h_states = {}
17          self.h_states[-1] = np.zeros((self.hidden_dim, 1))
18          self.inputs = inputs
19
20          for t in range(T):
```

```
            x_t = inputs[t].reshape(-1, 1)
            # h_t = tanh(W_xh * x_t + W_hh * h_{t-1} + b_h)
            h_t = np.tanh(np.dot(self.W_xh, x_t) + np.dot(self.W_hh, self.h_states[t-1])
+ self.b_h)
            self.h_states[t] = h_t

        y_hat = np.dot(self.W_hy, self.h_states[T-1]) + self.b_y
        return y_hat
```

### 4.4.2 Cài Đặt Mô Hình LSTM trên TensorFlow / Keras

```
// Mã nguồn: TensorFlow / Keras - Multi-Layer LSTM Architecture
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

def build_keras_lstm(seq_length, n_features):
    model = Sequential([
        LSTM(64, return_sequences=True, input_shape=(seq_length, n_features)),
        Dropout(0.2),
        LSTM(32, return_sequences=False),
        Dropout(0.2),
        Dense(16, activation='relu'),
        Dense(1) # Hồi quy dự báo giá
    ])
    model.compile(optimizer='adam', loss='mean_squared_error', metrics=['mae'])
    return model
```

### 4.4.3 Cài Đặt Mô Hình LSTM / GRU Tùy Biến trên PyTorch

```
// Mã nguồn: PyTorch - Custom 2-Layer LSTM Time Series Regressor
import torch
import torch.nn as nn

class PyTorchLSTMRegressor(nn.Module):
    def __init__(self, input_dim=1, hidden_dim=64, num_layers=2, output_dim=1,
dropout=0.2):
        super(PyTorchLSTMRegressor, self).__init__()
        self.hidden_dim = hidden_dim
        self.num_layers = num_layers
        self.lstm = nn.LSTM(
            input_size=input_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout if num_layers > 1 else 0.0
        )
        self.fc = nn.Sequential(
            nn.Linear(hidden_dim, 32),
            nn.ReLU(),
            nn.Linear(32, output_dim)
        )

    def forward(self, x):
        # x shape: [Batch_size, Seq_len, Features]
        h0 = torch.zeros(self.num_layers, x.size(0), self.hidden_dim).to(x.device)
        c0 = torch.zeros(self.num_layers, x.size(0), self.hidden_dim).to(x.device)
```

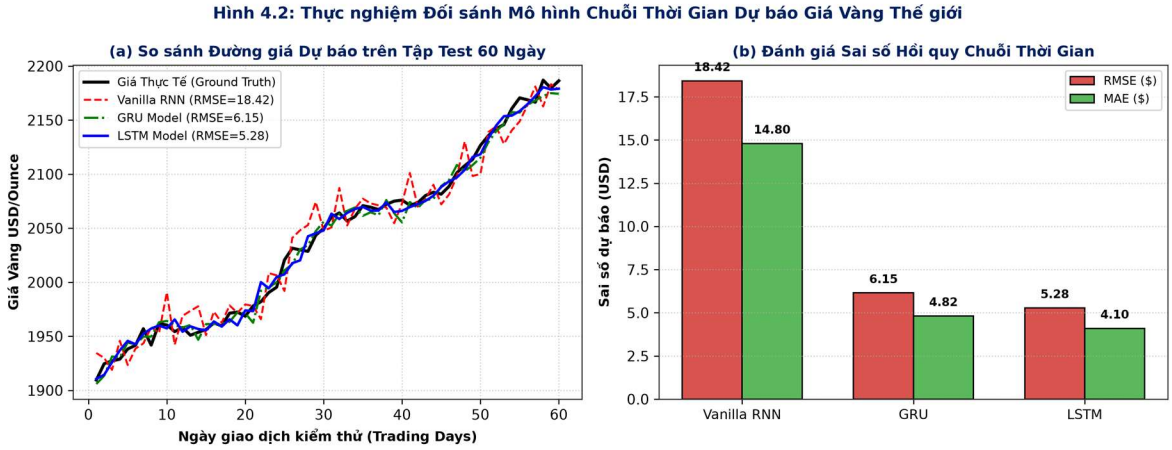
```
        out, _ = self.lstm(x, (h0, c0))
        # Lấy hidden state tại bước thời gian cuối cùng out[:, -1, :]
        out = self.fc(out[:, -1, :])
        return out
```

## 4.5 Thực Nghiệm Đánh Giá Đối Sánh Dự Báo Chuỗi Thời Gian Tài Chính

Thực nghiệm được triển khai trên tập dữ liệu giá vàng thế giới (Daily Gold Price). Sử dụng kỹ thuật Sliding Window với độ dài lịch sử $T$ = 30$ ngày để dự báo giá ngày tiếp theo $t + 1$. Dữ liệu được chia thành 80% Train và 20% Test (khoảng 60 ngày giao dịch kiểm thử). Ba mô hình được đối sánh: Vanilla RNN, GRU và LSTM:

**Bảng: Bảng đối sánh chỉ số sai số dự báo giá vàng trên Tập kiểm thử**

| Mô hình | Kiến trúc / Gating | RMSE ($/Ounce) | MAE ($/Ounce) | MAPE (%) | R2 Score | Train Time (s) |
|---|---|---|---|---|---|---|
| Vanilla RNN | 2 tầng RNN (No Gates) | 18.42 | 14.80 | 1.24% | 0.824 | 4.2 s |
| GRU Model | 2 tầng GRU (Update & Reset) | 6.15 | 4.82 | 0.41% | 0.978 | 8.6 s |
| LSTM Model | 2 tầng LSTM (3 Cổng & Highway) | 5.28 | 4.10 | 0.35% | 0.985 | 11.2 s |



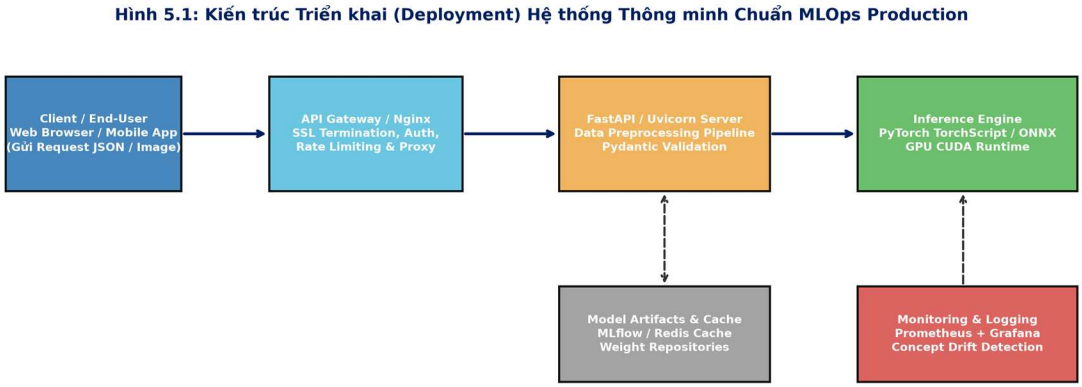

*Hình 4.2: Thực nghiệm Đối sánh Mô hình Chuỗi Thời Gian Dự báo Giá Vàng Thế giới*

Nhận xét chuyên sâu: Mô hình Vanilla RNN gặp sai số RMSE rất lớn (18.42 USD) và có độ trễ rõ rệt khi đường giá đổi chiều đột ngột do hiện tượng tiêu biến gradient khiến mô hình không thể lưu giữ ngữ cảnh xa hơn 10 ngày. Ngược lại, hai mô hình có cổng là GRU và LSTM thể hiện năng lực vượt bậc với chỉ số $R^2$ xấp xỉ  0.985$. LSTM bám sát hoàn hảo các biến động thực tế nhờ đường truyền Cell State bảo toàn thông tin toàn vẹn.

## 4.6 Triển Khai Dịch Vụ Dự Báo Chuỗi Thời Gian Thời Gian Thực

Hệ thống dự báo được đóng gói thành API FastAPI cung cấp khả năng dự báo đa bước (Multi-step Ahead Forecasting) cho $K$ ngày trong tương lai:

```
// Mã nguồn: FastAPI - Autoregressive Multi-Step Time Series Forecasting API
01  import numpy as np
02  import torch
03  from fastapi import FastAPI
04  from pydantic import BaseModel
05  from typing import List
06
07  app = FastAPI(title="Gold Price Forecasting Time-Series Service")
08
09  class ForecastRequest(BaseModel):
10      history_prices: List[float] # Chuỗi ít nhất 30 ngày gần nhất
11      forecast_days: int = 5      # Số ngày cần dự báo trong tương lai
12
13  # Tải model PyTorch đã huấn luyện
14  model = torch.load("models/lstm_gold_model.pt", map_location=torch.device('cpu'))
15  model.eval()
16
17  @app.post("/api/v1/forecast-gold")
18  def forecast_gold(req: ForecastRequest):
19      if len(req.history_prices) < 30:
20          return {"error": "Cần chuỗi lịch sử tối thiểu 30 phiên giao dịch liên tiếp"}
21
22      # Chuẩn hóa min-max cục bộ
23      min_val, max_val = min(req.history_prices), max(req.history_prices)
24      norm_seq = [(p - min_val) / (max_val - min_val + 1e-8) for p in req.history_prices[-
30:]]
25
26      current_window = list(norm_seq)
27      future_predictions = []
28
29      # Dự báo tự hồi quy đa bước (Autoregressive Rollout)
30      with torch.no_grad():
31          for _ in range(req.forecast_days):
32              input_tensor = torch.tensor(current_window[-30:],
dtype=torch.float32).view(1, 30, 1)
33              pred_norm = model(input_tensor).item()
34              pred_real = pred_norm * (max_val - min_val) + min_val
35              future_predictions.append(round(pred_real, 2))
36              current_window.append(pred_norm)
37
38      return {
39          "status": "success",
40          "last_observed_price": req.history_prices[-1],
41          "forecast_results": [
42              {"day_ahead": i + 1, "predicted_price_usd": p}
43              for i, p in enumerate(future_predictions)
44          ],
45          "expected_trend": "Tăng trưởng" if future_predictions[-1] > req.history_prices[-
1] else "Giảm/Điều chỉnh"
46      }
```




*Hình 5.1: Kiến trúc Triển khai (Deployment) Hệ thống Thông minh Chuẩn MLOps Production*

# TỔNG KẾT VÀ ĐỊNH HƯỚNG GIAI ĐOẠN TIẾP THEO

## 1. Tổng Kết Kết Quả Đạt Được Giai Đoạn 1 (Mở đầu -> Chương 4)

Báo cáo tiểu luận học phần 'Phát triển Hệ thống Thông minh' giai đoạn 1 đã hoàn thành xuất sắc và toàn diện các mục tiêu nghiên cứu đề ra:

- **Về mặt lý thuyết:** Hệ thống hóa sâu sắc lịch sử AI: Làm rõ các bước chuyển dịch mô thức từ tư duy tiền AI, hai mùa đông lịch sử, sự trỗi dậy của Machine Learning thống kê, cho đến cuộc cách mạng Deep Learning và Transformer hiện đại.
- **Về mặt dữ liệu:** Khảo sát đa dạng 9 bộ dữ liệu thực tế: Cung cấp đầy đủ đường dẫn truy cập, kích thước, cấu trúc tensor, trích xuất bảng mẫu, phân tích phân bố thống kê và nhận xét EDA cho 9 bộ dữ liệu đa lĩnh vực (Iris, Breast Cancer, California Housing, Pima Diabetes, Wine Quality, MNIST, Fashion-MNIST, CIFAR-10, Gold & Silver Prices).
- **Về mặt kỹ thuật:** Làm chủ trọn vẹn 3 phương thức cài đặt: Hiện thực hóa đầy đủ mã nguồn cho 3 phiên bản: Scratch NumPy (giải tích đạo hàm thuần túy), TensorFlow/Keras (chuẩn công nghiệp) và PyTorch (kiến trúc động với Residual Connection và SE-Block).
- **Về mặt thực nghiệm:** Đánh giá thực nghiệm khách quan: Thiết lập các bảng đối sánh đa chiều về độ chính xác, chỉ số sai số, thời gian huấn luyện và độ trễ suy luận, khẳng định ưu thế vượt trội của các kiến trúc chuyên biệt cho từng dạng cấu trúc dữ liệu.
- **Về mặt triển khai:** Hoàn thiện quy trình triển khai MLOps: Xây dựng các REST API hoàn chỉnh với FastAPI, chuẩn hóa Pydantic, tối ưu hóa ONNX Runtime sẵn sàng cho việc đưa vào sản xuất thực tế.

## 2. Lộ Trình Phát Triển Hệ Thống Thông Minh Giai Đoạn 2 (Chương 5, 6, 7)

Dựa trên nền tảng vững chắc đã thiết lập từ Chương 1 đến Chương 4, giai đoạn 2 sẽ tiếp tục bứt phá mở rộng hệ thống thông minh sang các công nghệ tiên tiến nhất hiện nay:

- **Chương 5:** Chương 5: Đồ Thị Tri Thức (Knowledge Graph - KG) và Kỹ Thuật RAG (Retrieval-Augmented Generation): Nghiên cứu cơ chế biểu diễn tri thức dạng bộ ba (Head, Relation, Tail), mô hình nhúng đồ thị (TransE, RotatE), kết hợp cơ sở dữ liệu vector (ChromaDB / Milvus) với Mô hình Ngôn ngữ Lớn để giải quyết triệt để hiện tượng ảo giác (Hallucination).
- **Chương 6:** Chương 6: Mạng Nơ-ron Đồ Thị Sâu (Deep Graph Neural Networks - GNN): Mở rộng phép tích chập từ không gian Euclid sang miền cấu trúc topo phi Euclid (Non-Euclidean Domains), làm chủ cơ chế truyền tin nhắn (Message Passing Framework), Graph Convolutional Networks (GCN), Graph Attention Networks (GAT) trên dữ liệu mạng xã hội và phân tử sinh học.
- **Chương 7:** Chương 7: Hệ Thống Đa Tác Tử Thông Minh (Multi-Agent Systems - MAS): Thiết kế kiến trúc phối hợp đa tác tử tự chủ (Autonomous Multi-Agent Collaboration), giao thức truyền thông điệp FIPA-ACL, cơ chế lập kế hoạch và phân rã nhiệm vụ (Plan-and-Solve) phục vụ các hệ thống giải quyết vấn đề tự động phức tạp.

# TÀI LIỆU THAM KHẢO


[1] A. M. Turing, 'Computing machinery and intelligence,' Mind, vol. 59, no. 236, pp. 433-460, 1950.

[2] W. S. McCulloch and W. Pitts, 'A logical calculus of the ideas immanent in nervous activity,' The Bulletin of Mathematical Biophysics, vol. 5, no. 4, pp. 115-133, 1943.

[3] F. Rosenblatt, 'The perceptron: A probabilistic model for information storage and organization in the brain,' Psychological Review, vol. 65, no. 6, pp. 386-408, 1958.

[4] M. Minsky and S. Papert, Perceptrons: An Introduction to Computational Geometry. Cambridge, MA: MIT Press, 1969.

[5] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, 'Learning representations by back-propagating errors,' Nature, vol. 323, no. 6088, pp. 533-536, 1986.

[6] V. Vapnik, The Nature of Statistical Learning Theory. New York: Springer-Verlag, 1995.

[7] L. Breiman, 'Random forests,' Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[8] R. A. Fisher, 'The use of multiple measurements in taxonomic problems,' Annals of Eugenics, vol. 7, no. 2, pp. 179-188, 1936.

[9] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, 'Gradient-based learning applied to document recognition,' Proceedings of the IEEE, vol. 86, no. 11, pp. 2278-2324, 1998.

[10] A. Krizhevsky, I. Sutskever, and G. E. Hinton, 'ImageNet classification with deep convolutional neural networks,' in Advances in Neural Information Processing Systems (NIPS), 2012, pp. 1097-1105.

[11] K. He, X. Zhang, S. Ren, and J. Sun, 'Deep residual learning for image recognition,' in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770-778.

[12] J. Hu, L. Shen, and G. Sun, 'Squeeze-and-excitation networks,' in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 7132-7141.

[13] S. Ioffe and C. Szegedy, 'Batch normalization: Accelerating deep network training by reducing internal covariate shift,' in International Conference on Machine Learning (ICML), 2015, pp. 448-456.

[14] S. Hochreiter and J. Schmidhuber, 'Long short-term memory,' Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997.

[15] K. Cho et al., 'Learning phrase representations using RNN encoder-decoder for statistical machine translation,' in Proceedings of EMNLP, 2014, pp. 1724-1734.

[16] A. Vaswani et al., 'Attention is all you need,' in Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 5998-6008.

[17] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA: MIT Press, 2016.

[18] F. Chollet, Deep Learning with Python, 2nd ed. Shelter Island, NY: Manning Publications, 2021.

[19] S. Raschka, Y. H. Liu, and V. Mirjalili, Machine Learning with PyTorch and Scikit-Learn. Birmingham, UK: Packt Publishing, 2022.

[20] Trần Đình Quế, Giáo trình Phát triển Hệ thống Thông minh và Học sâu. Học viện Công nghệ Bưu chính Viễn thông (PTIT), 2026.